



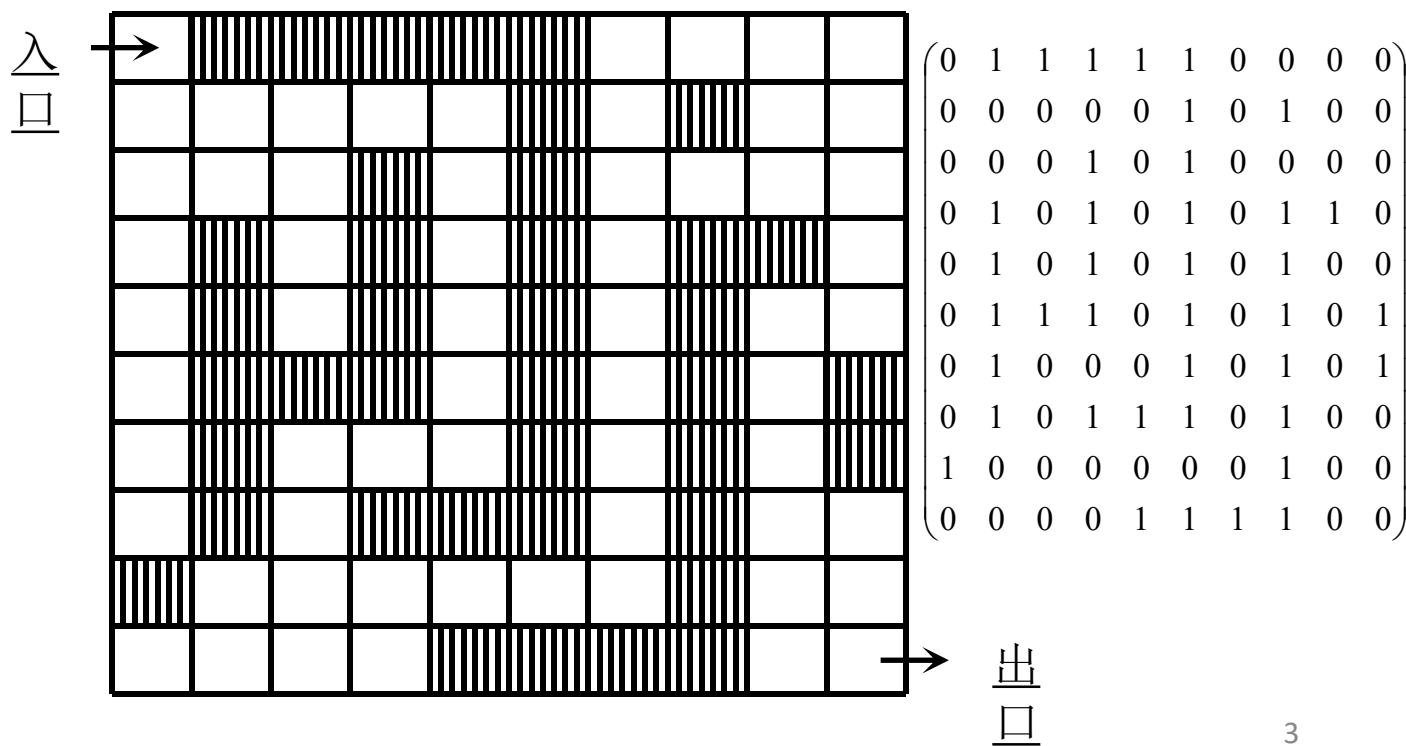
算法设计与分析

讲授内容：回溯与分支限界

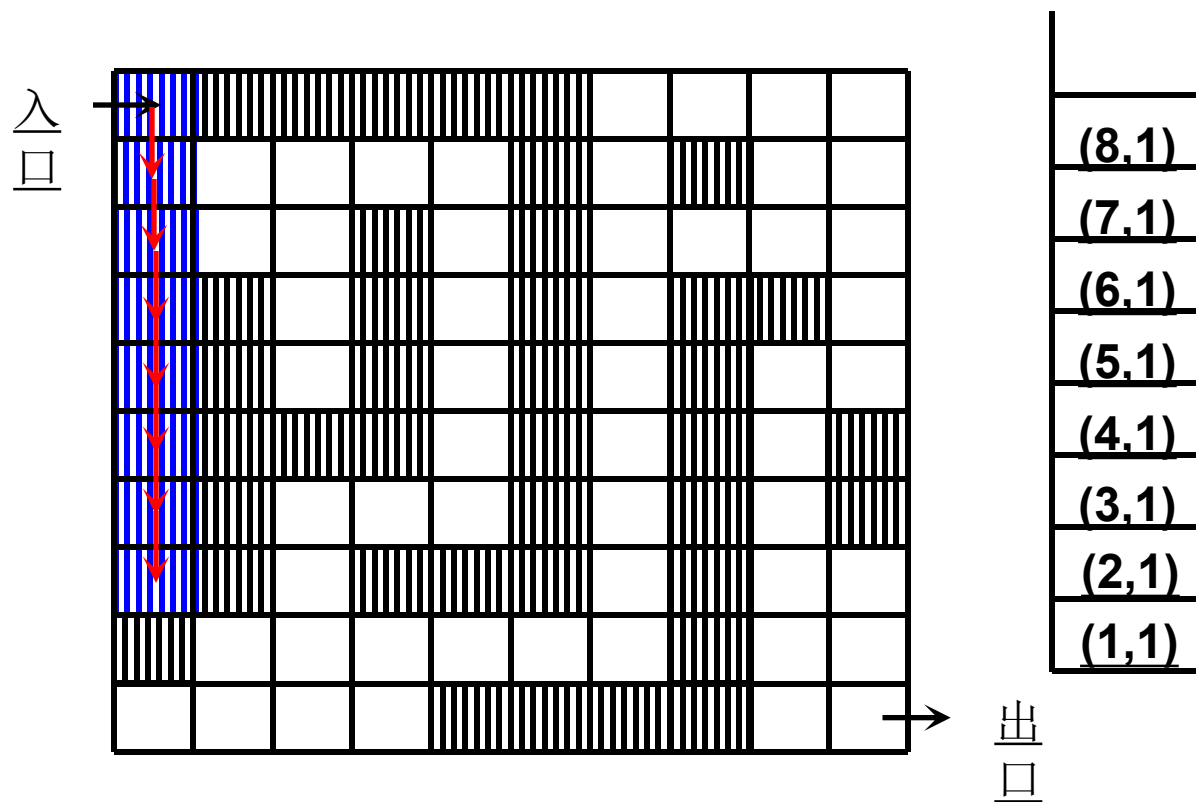
2025年11月10日

- 引例
- 回溯算法的基本思想和适用条件
- 回溯算法的设计步骤
- 应用实例
- 回溯算法的改进途径

引例：迷宫老鼠问题



引例：迷宫老鼠问题



- 类似枚举的搜索尝试过程，主要是在搜索尝试过程中寻找问题的解，当探索到某一步时，发现原先选择不优或达不到目标，就退回一步重新选择，尝试别的路径。
 - “走不通就退回再走”
- 满足回溯条件的某个状态的点称为“回溯点”

□ 回溯法：通常以深度优先的方式系统地搜索问题的解的算法

□ 使用场合

➤ 对于许多问题，当需要找出它的解的集合或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。

➤ 这种方法适用于解一些组合数相当大的问题，具有“通用解法”之称。

□ 回溯法的基本做法

➤ 是搜索，或是一种组织得井井有条的，能避免不必要搜索的穷举式搜索法。

具体做法

□ 系统性

回溯法在问题的解空间树中，通常按深度优先策略，从根结点出发搜索解空间树。

□ 跳跃性

算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解。如果肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。

回溯法与穷举查找是一样的吗？

□ 可以把回溯和分支限界看成是穷举法的一个改进。

该方法至少可以对某些组合难题的较大**实例求解**。

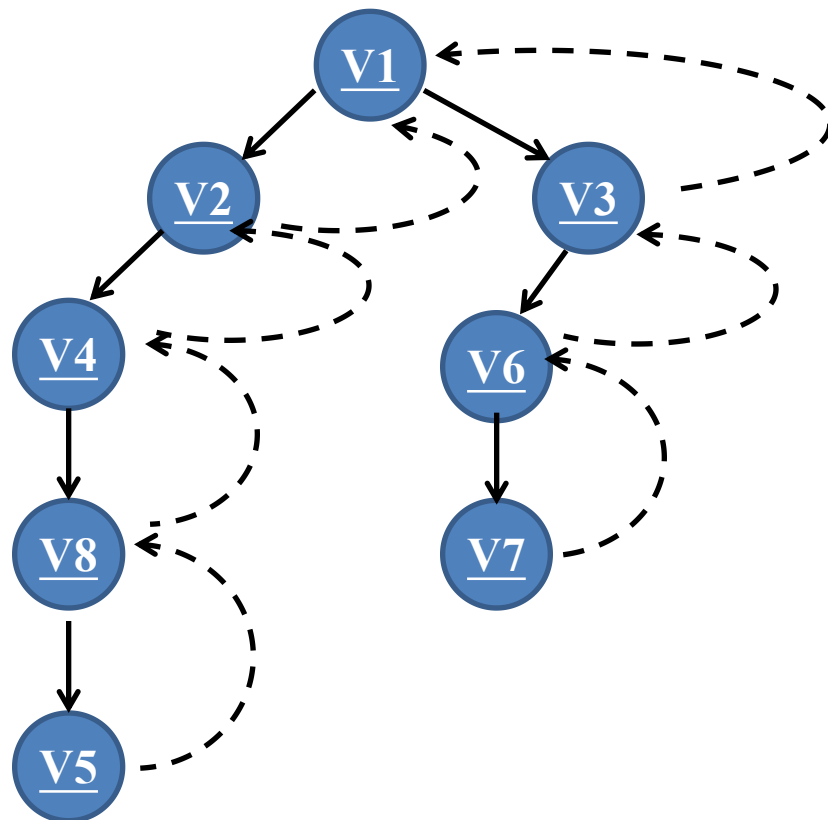
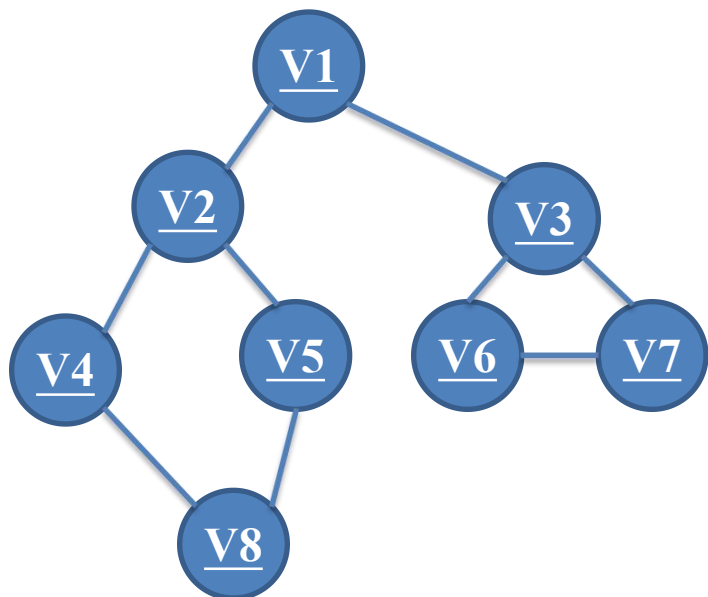
□ 不同点

➤ 每次只构造候选解的一个部分

➤ 然后评估这个部分构造解：如果加上剩余的分量也不可能求得一个解，就绝对不会生成剩下的分量

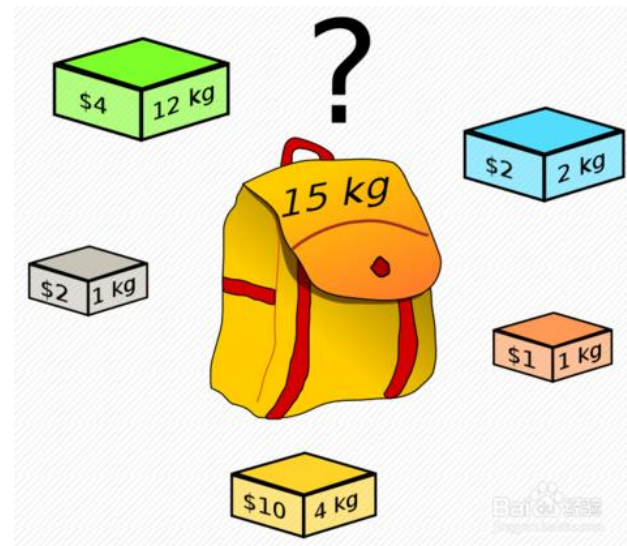
回溯法引言

回溯算法：图的深度优先遍历



几个典型例子：0-1背包问题

- 例：0-1背包问题
- 给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 c 。问应该如何选择装入背包中的物品，使得装入背包中物品的总价值最大？



例：0-1背包问题

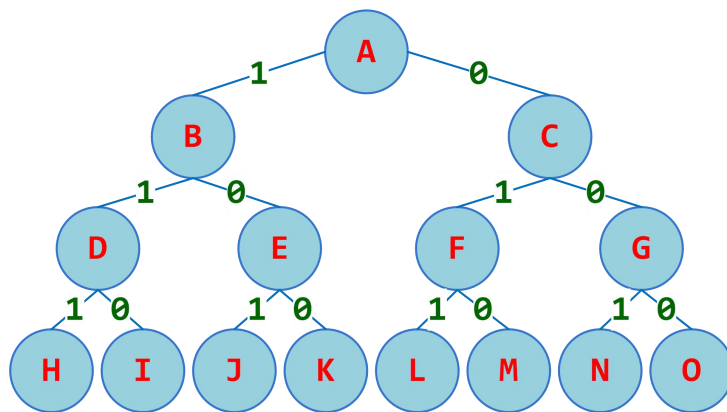
- $n=3$, $C=30$, $w=\{16, 15, 15\}$, $v=\{45, 25, 25\}$

- 开始时,

- $C_r=C=30$, $V=0$

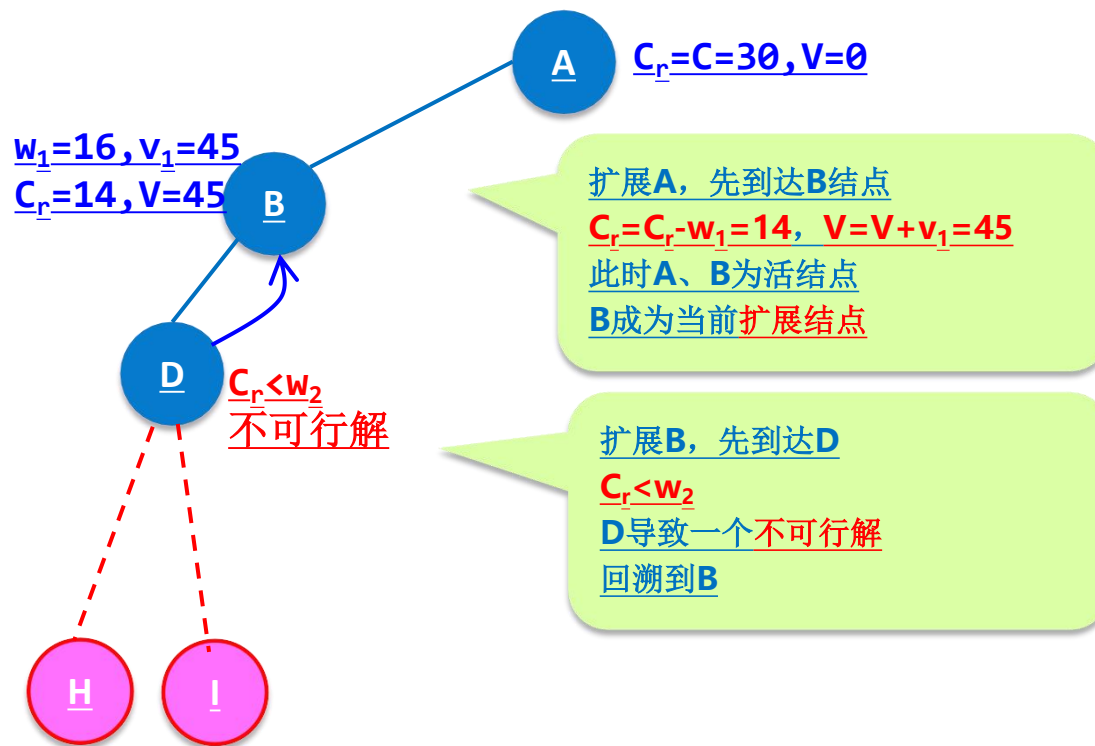
C 为容量, C_r 为剩余空间, V 为价值。

- A 为唯一活结点, 也是当前扩展结点。



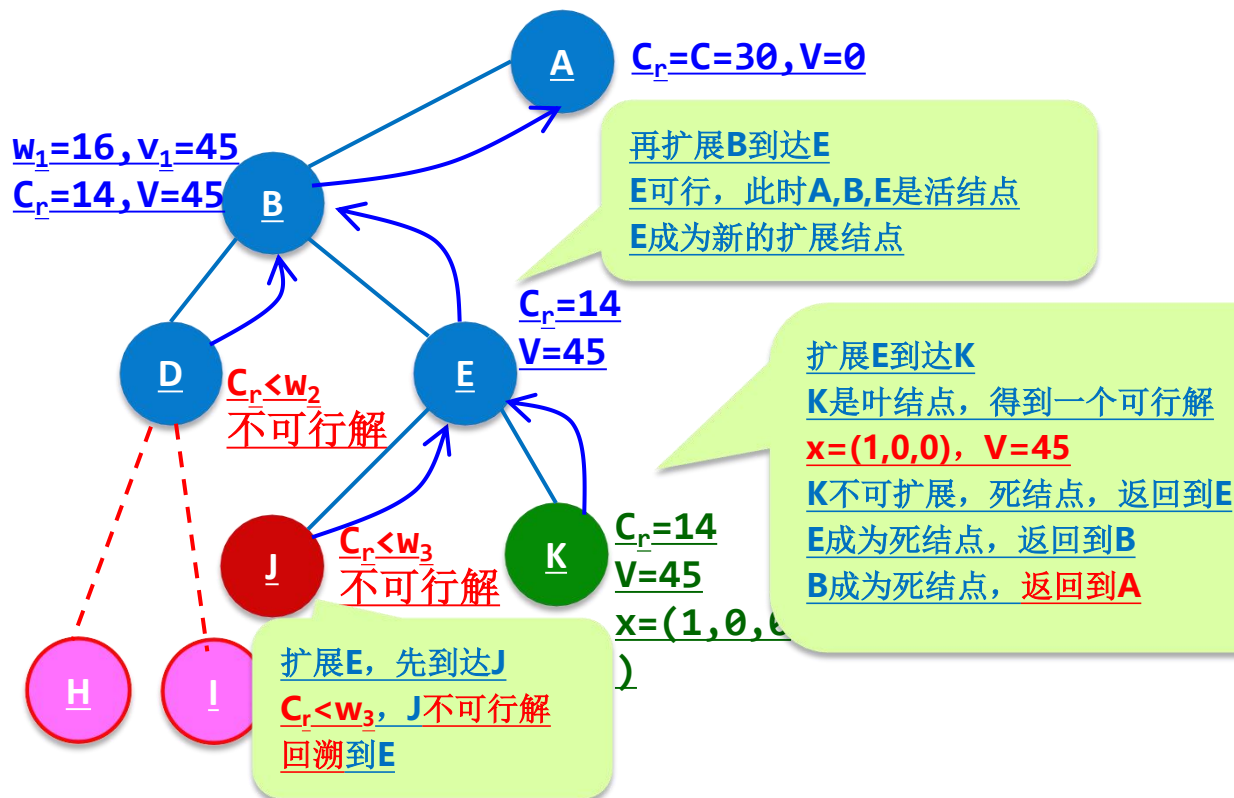
回溯法的算法框架

- $n=3$, $C=30$, $w=\{16,15,15\}$, $v=\{45,25,25\}$



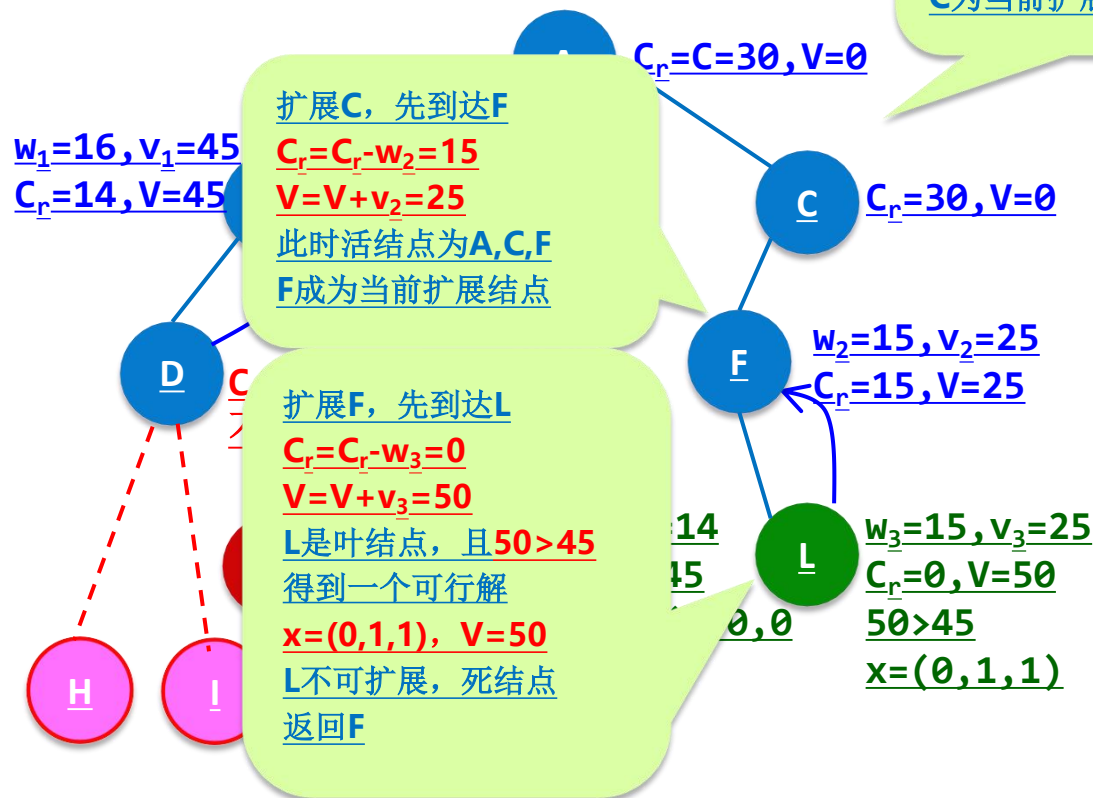
回溯法的算法框架

- $n=3$, $C=30$, $w=\{16,15,15\}$,
 $v=\{45,25,25\}$



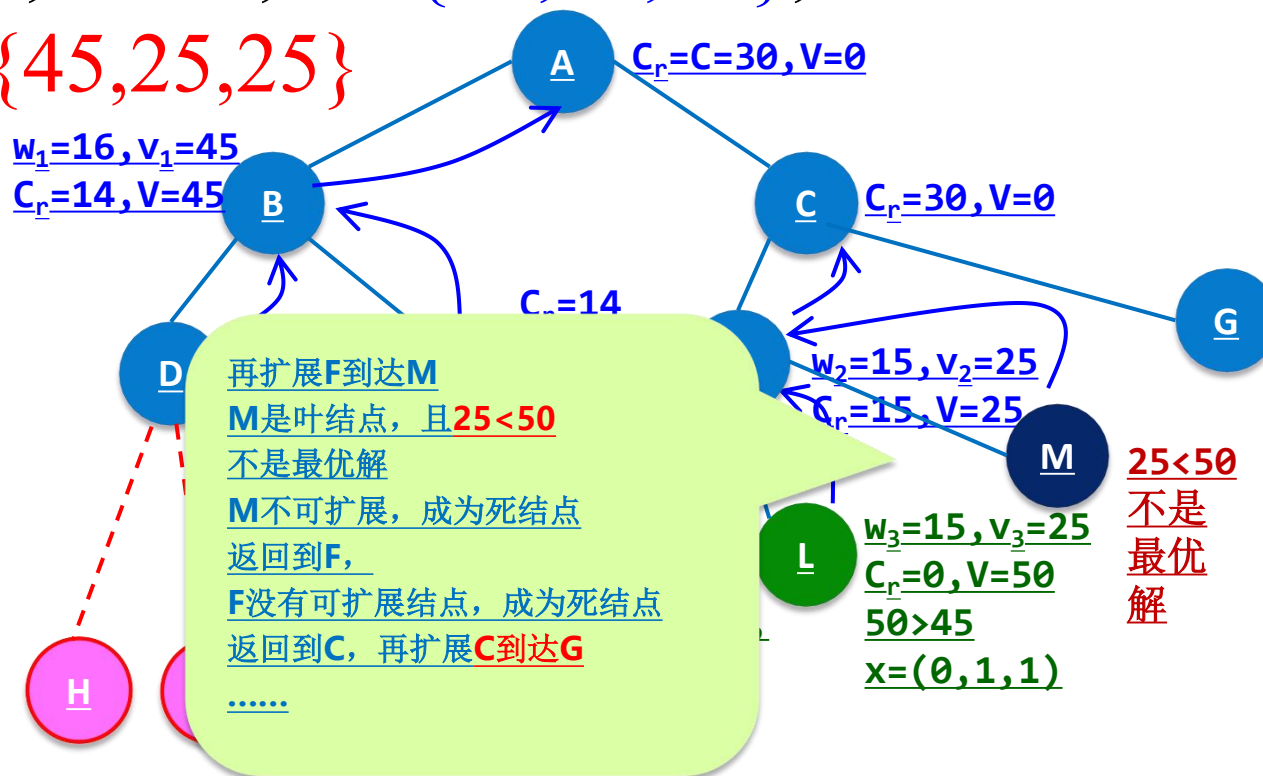
回溯法的算法框架

- $n=3$, $C=30$, $w=\{16,15,15\}$,
 $v=\{45,25,25\}$



回溯法的算法框架

- $n=3$, $C=30$, $w=\{16,15,15\}$,
 $v=\{45,25,25\}$

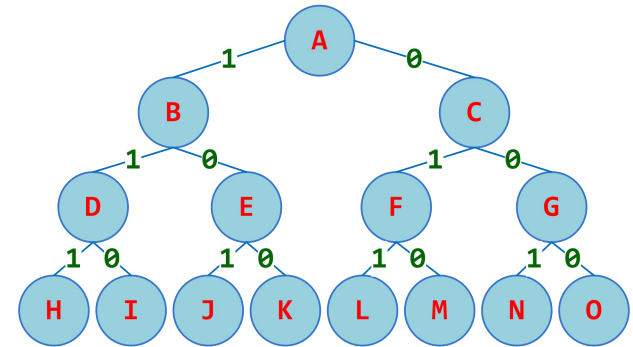


回溯法的算法框架

- $n=3$, $C=30$, $w=\{16,15,15\}$,
 $v=\{45,25,25\}$

- 扩展到G以后.....

- $C_r=30$, $V=0$,
活结点为A、C、G, G为当前扩展结点
- 扩展G, 先到达N, N是叶结点, 且 $25 < 50$, 不是
最优解, 又N不可扩展, 返回到G
- 再扩展G到达O, O是叶结点, 且 $0 < 50$, 不是最
优解, 又O不可扩展, 返回到G
- G没有可扩展结点, 成为死结点, 返回到C
- C没有可扩展结点, 成为死结点, 返回到A
- A没有可扩展结点, 成为死结点, 算法结束
- 最优解 $X=(0,1,1)$, 最优值50。



- 算法基本过程

将待求解问题看做一个解空间树,问题的解可以表示为 $\mathbf{X} = (x_1, x_2, \dots, x_n)$,然后利用深度优先搜索逐步确定每一个解 x_i ,当搜索到树的叶子结点时,就得到问题的一个解 X_i .

当然这个解不一定是最优解,在将整个解空间树搜索完之后,通比较得到的每个 X_i ,便可以得到最优解.

- 在搜索的过程中,问题的解 $\mathbf{X}$ 需要满足约束条件 $\mathbf{P}(\mathbf{X})$ 。在搜索到一个结点的时候发现当前结点不满足约束条件,则放弃向下搜索,即不再搜索该结点的子结点,而是回溯到上一个结点继续搜索.

- 由于在搜索过程中,放弃了一些没有必要搜索的结点,整个算法的效率就提高了
- 问题: 为什么能够放弃?
 - ✓ 如果当前结点不满足约束条件,能够推导出它的子结点也不满足约束条件,那么就可以放弃搜索它的子结点.
 - ✓ 多米诺性质.

回溯算法的适用条件

- 在搜索的过程中,问题的解 X 需要满足约束条件 $P(X)$
- 在搜索到一个结点的时候发现当前结点不满足约束条件,则放弃向下搜索,即不再搜索该结点的子结点,而是回溯到上一个结点继续搜索.
- 多米诺性质:
 - 设 $X=\langle x_1, x_2, \dots, x_n \rangle$ 是问题的解, X_i 和 X_{i+1} 分别是搜索到第 i 层和第 $i+1$ 层的解, 其中 $X_i=\langle x_1, x_2, \dots, x_i \rangle$, $X_i \subseteq X$. $P(x_1, x_2, \dots, x_i)$ 为真表示向量 $X_i=\langle x_1, x_2, \dots, x_i \rangle$ 满足某个性质(n 后问题中 i 个皇后放置在彼此不能攻击的位置). 如果 $P(X_{i+1}) \rightarrow P(X_i)$, $0 < i < n$, 则该问题满足多米诺性质
 - 逆否命题: 如果子结点满足约束条件能够推导出其父结点满足约束条件

- 1. 背包问题

约束条件如下，其中 n 是物品的数量， x_i 表示是否选择该物品， w_i 表示物品的重量， C 为背包容量。

$$\begin{cases} \sum_{i=1}^n x_i * w_i \leq C, 0 < i \leq n \\ x_i \in \{0, 1\}, 0 < i \leq n \\ w_i > 0, 0 < i \leq n \end{cases}$$

多米诺性质的证明如下：

$$\text{设 } X_i = \sum_{k=1}^i x_k * w_k, X_{i+1} = \sum_{k=1}^{i+1} x_k * w_k$$

$$\because X_{i+1} \leq C, w_k > 0, x_k \in \{0, 1\}$$

$$\therefore X_i < X_{i+1} \leq C$$

- 2. 不等式的整数解

求解不等式 $5 * x_1 + 4 * x_2 - x_3 \leq 10$, $1 \leq x_i \leq 3$, $i = 1, 2, 3$ 的整数解.

□ 这个问题不满足多米诺性质: $P(X_3)$ 满足约束不能够推导出 $P(X_2)$ 满足约束

变换: 令 $x_3 = 3 - x'_3$,

$\Rightarrow 5 * x_1 + 4 * x_2 + x'_3 \leq 13$, $1 \leq x_1, x_2 \leq 3$, $0 \leq x'_3 \leq 2$

- (1)适用问题：求解搜索问题和优化问题
- (2)搜索空间：树，结点对应部分解向量，树叶对应可行解
- (3)搜索过程：采用系统的方法隐含遍历搜索树
- (4)搜索策略：深度优先，宽深结合等
- (5)结点分支判定条件：
 - 满足约束条件---分支扩张解向量
 - 不满足约束条件，回溯到该结点的父结点
- (6)结点状态：动态生成
 - 白结点(尚未访问);
 - 灰结点(正在访问该结点为根的子树);
 - 黑结点(该结点为根的子树遍历完成)
- (7)存储：当前路径

(1) 定义搜索问题的解向量和每个分量的取值范围

解向量为 $\langle x_1, x_2, \dots, x_n \rangle$

确定 x_i 的可能取值的集合为 $X_i, i = 1, 2, \dots, n$

(2) 当 $x_1, x_2, \dots, x_{k-1}$ 确定以后计算 x_k 取值集合 $S_k, S_k \subseteq X_k$

(3) 确定结点儿子的排列规则

(4) 判断是否满足多米诺性质

(5) 搜索策略----深度优先等

(6) 确定每个结点分支约束条件

(7) 确定存储搜索路径的数据结构

问题的解空间

□ 问题的解向量

回溯法希望一个问题的解能够表示成一个 n 元式
 $(x_1, x_2, \dots, x_n)$ 的形式。

□ 显约束

对分量 x_i 的取值限定。

□ 隐约束

为满足问题的解而对不同分量之间施加的约束。

解空间 (Solution Space)

□ 对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意：同一问题可有多种表示，有些表示更简单，所需状态空间更小（存储量少，搜索方法简单）。

例如

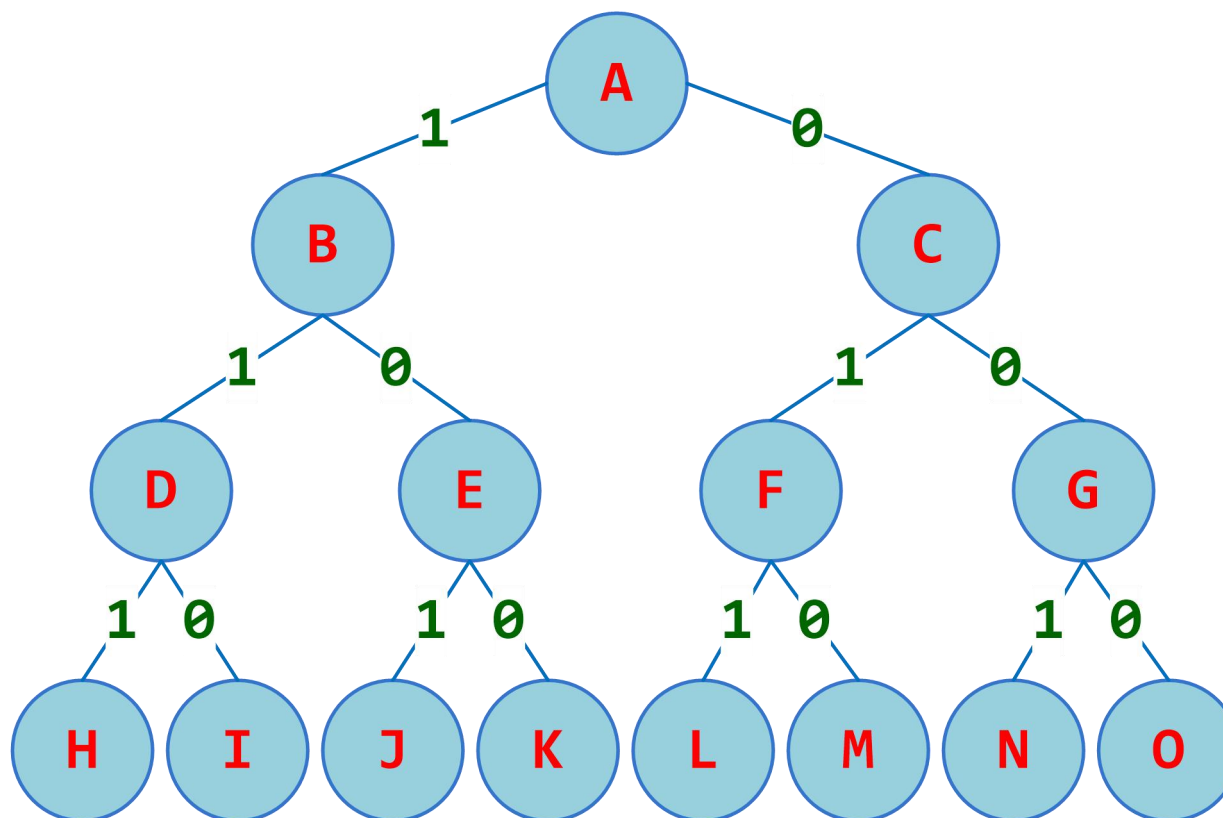
对于有 n 种可选物品的0-1背包问题

- ✓ 解空间由 2^n 个长度为 n 的0-1向量组成
- ✓ $n=3$ 时，解空间为 $\{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)\}$

用完全二叉树表示的解空间

- ✓ 边上的数字给出了向量 x 中第 i 个分量的值 x_i
- ✓ 根节点到叶节点的路径定义了解问题的一个解

回溯法的算法框架



对物品1的选择

对物品2的选择

对物品3的选择

回溯法的基本思想

□ 回溯法的基本步骤

- (1) 针对所给问题，定义问题的解空间；
- (2) 确定易于搜索的解空间结构；
- (3) 通常以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

□ 常用剪枝函数

- 用约束函数在扩展结点处剪去不满足约束的子树；
- 用限界函数剪去得不到最优解的子树。

空间复杂性

- 用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。
- 如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。
- 显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。

生成问题状态的基本方法

□ 扩展结点 (**E-结点, Expansion Node**)

一个正在产生儿子的结点称为扩展结点

□ 活结点 (**L-结点, Live Node**)

一个自身已生成但其儿子还没有全部生成的节点称做活结点

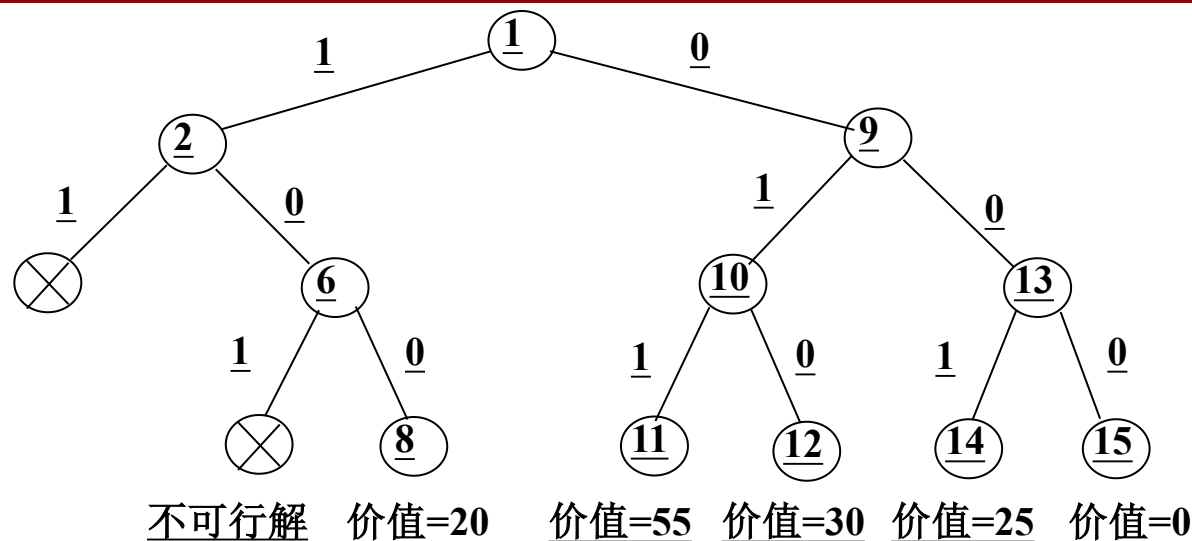
□ 死结点 (**D-结点, Dead Node**)

一个所有儿子已经产生的结点称做死结点

深度优先的问题状态生成法

- 如果对一个**扩展结点R**，一旦产生了它的一个**儿子C**，就把**C**当做新的**扩展结点**。
- 在**完成**对子树C（以C为根的子树）的**穷尽搜索**之后，将**R**重新变成**扩展结点**，继续生成R的下一个儿子（如果存在）。

回溯法的算法框架



- 当搜索到一个L结点时，就把这个L结点变成为E结点，继续向下搜索这个结点的儿子结点。
- 当搜索到一个D结点，而还未得到问题的最终解时，就向上回溯到它的父亲结点。
 - 如果这个父亲结点当前还是E结点，就继续搜索这个父亲结点的另一个儿子结点
 - 如果这个父亲结点随着所有儿子结点都已搜索完毕而变成D结点，就沿着这个父结点向上，回溯到它的祖父结点。
- 这个过程继续进行，直到找到满足问题的最终解。

回溯法

□ 为了避免生成那些不可能产生最佳解的问题状态，要不断地利用**限界函数(Bounding Function)**来处死（剪枝）那些实际上不可能产生所需解的活结点，以减少问题的计算量。

- 剪枝函数

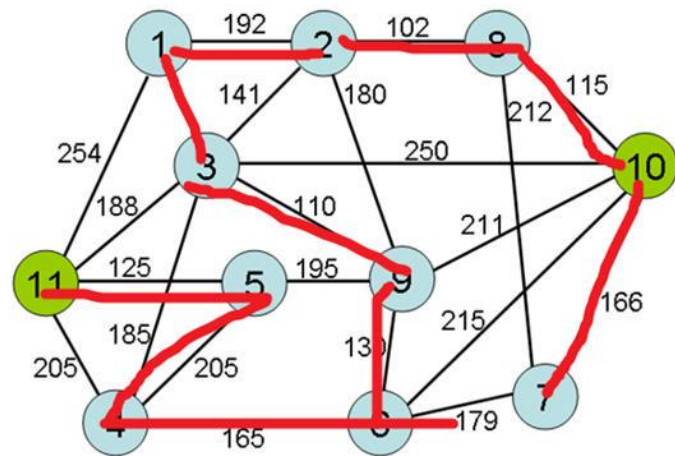
- **约束条件**剪去“不可行解”的子树

- **上界条件**剪去不含最优解的子树

保证算法搜索到的每个叶结点都是迄今为止找到的最优解

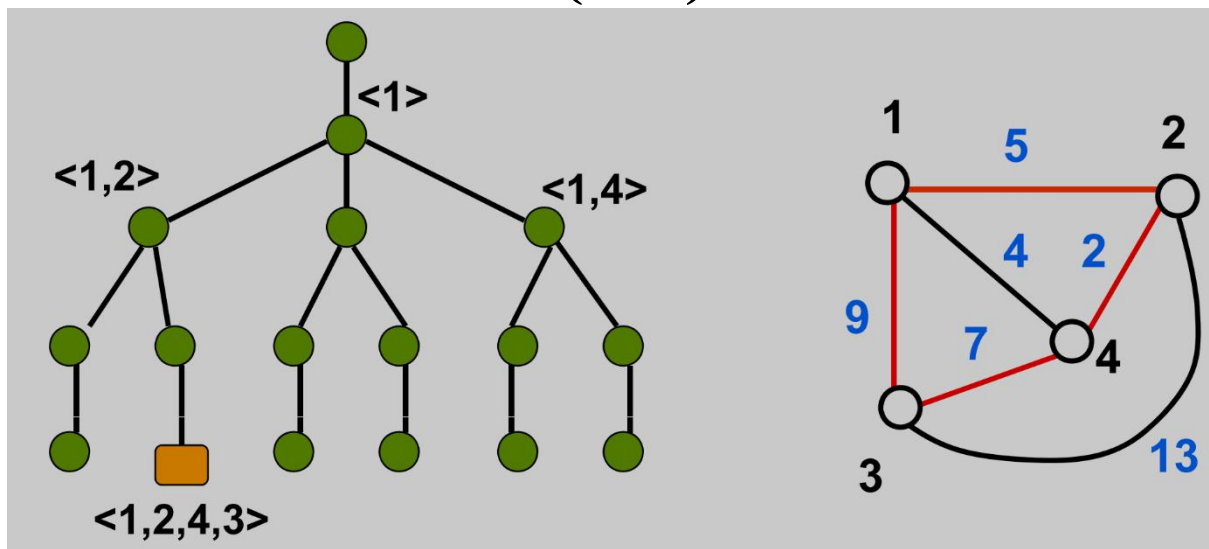
几个典型例子：旅行商问题

- 例：旅行商问题（又称货郎问题）
- 给定一系列城市和每对城市之间的距离，求解访问每一座城市一次并回到起始城市的最短回路。



几个典型例子：旅行商问题

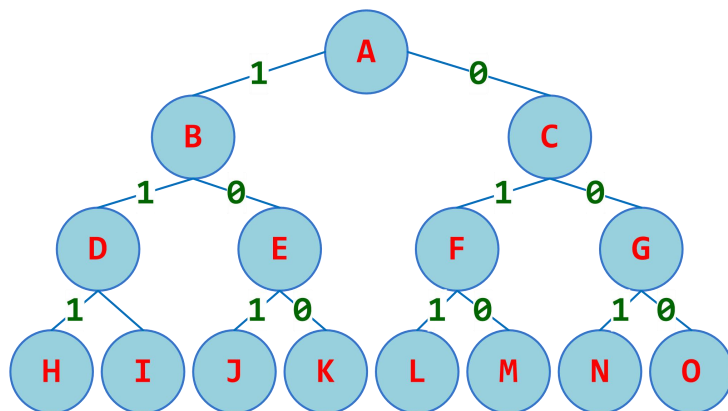
- 例：旅行商问题（又称货郎问题）
- 向量 $(x_1, x_2, \dots, x_n)$ 为巡回路线
- 搜索空间：排列树， $(n-1)!$ 片树叶



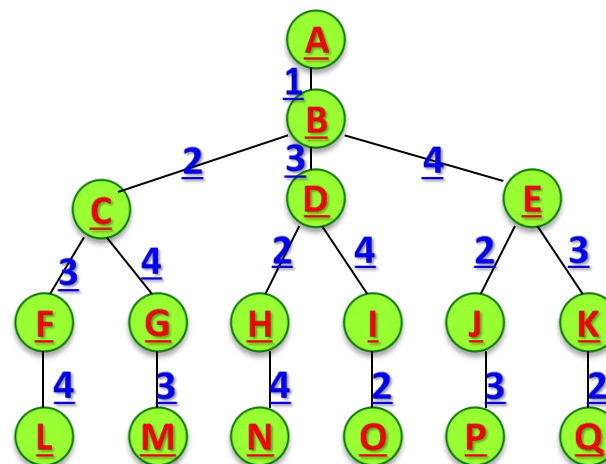
- $\langle 1, 2, 4, 3 \rangle$ 对应于巡回路线：1- $\rightarrow$ 2- $\rightarrow$ 4- $\rightarrow$ 3- $\rightarrow$ 1
- 路径长度：5+2+7+9=23

子集树与排列树

子集树



排列树



子集树与排列树

■ 子集树

- 从n个元素的集合S中找出S满足某种性质的子集，相应的解空间称为子集树。通常有 2^n 个叶结点，结点总数为 $2^{n+1}-1$ 。需 $\Omega(2^n)$ 计算时间。
- 如n个物品的0-1背包问题的解空间是一棵子集树。

■ 排列树

- 当所给问题的确定n个元素满足某种性质的排列时，相应的解空间树称为排列树。排列树通常有 $n!$ 个叶结点。因此遍历排列树需要 $\Omega(n!)$ 计算时间。
- TSP(旅行售货员问题)的解空间是一棵排列树。

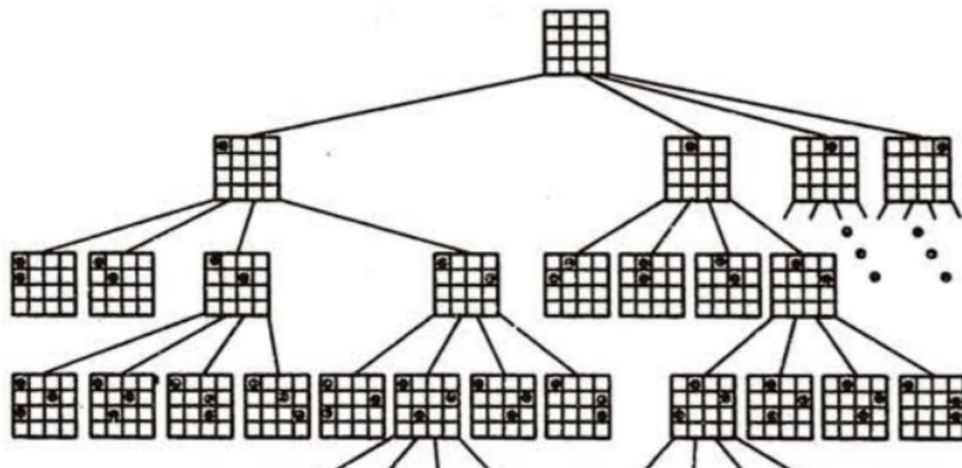
n 后问题

- 八皇后问题是十九世纪著名的数学家高斯于1850年提出的。
- 问题是：在 8×8 的棋盘上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上。
- **n 皇后问题**：即在 $n \times n$ 的棋盘上摆放 n 个皇后，使任意两个皇后都不能处于同一行、同一列或同一斜线上。

1			Q					
2					Q			
3							Q	
4		Q						
5						Q		
6	Q							
7			Q					
8				Q				
	1	2	3	4	5	6	7	8

◆ 例： n 后问题

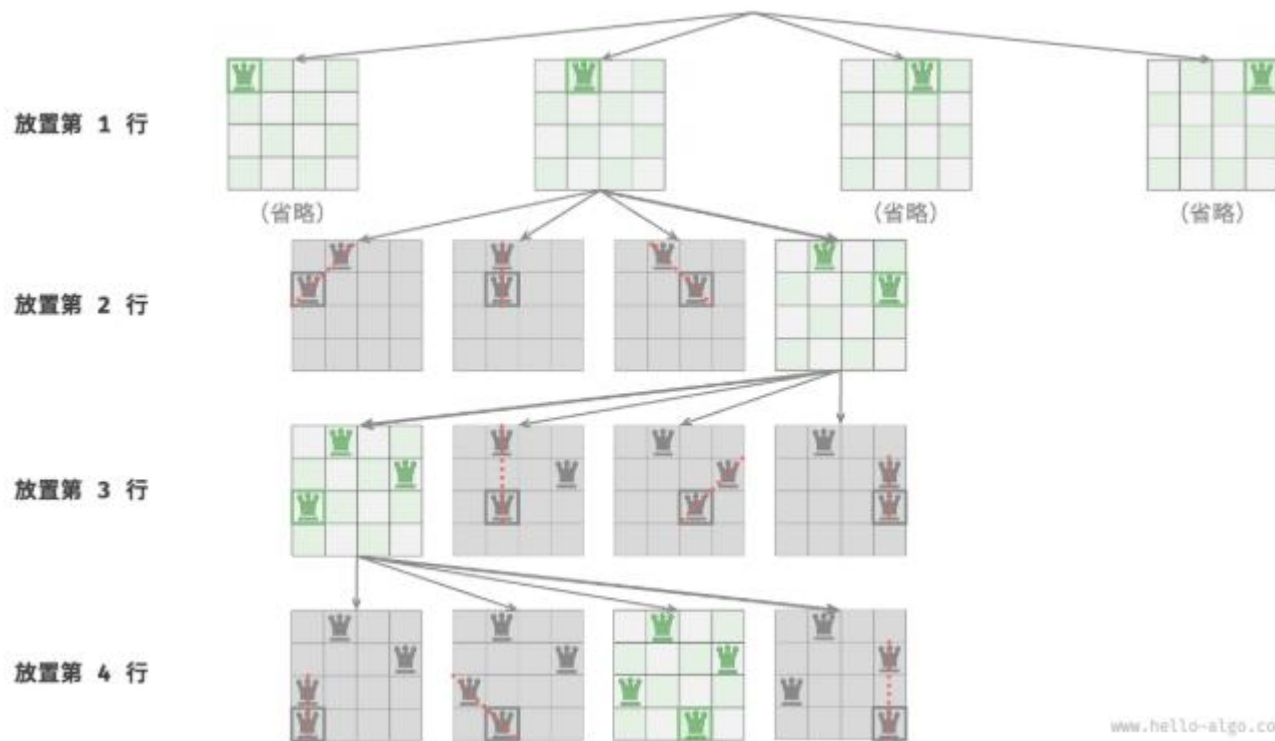
- 4后问题：解是一个4维向量， (x_1, x_2, x_3, x_4) （放置列号）
- 搜索空间：4叉树



- 8后问题：解是一个8维向量， $(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$
- 搜索空间：8叉树, 一个解： $\langle 1, 3, 5, 2, 4, 6, 8, 7 \rangle$

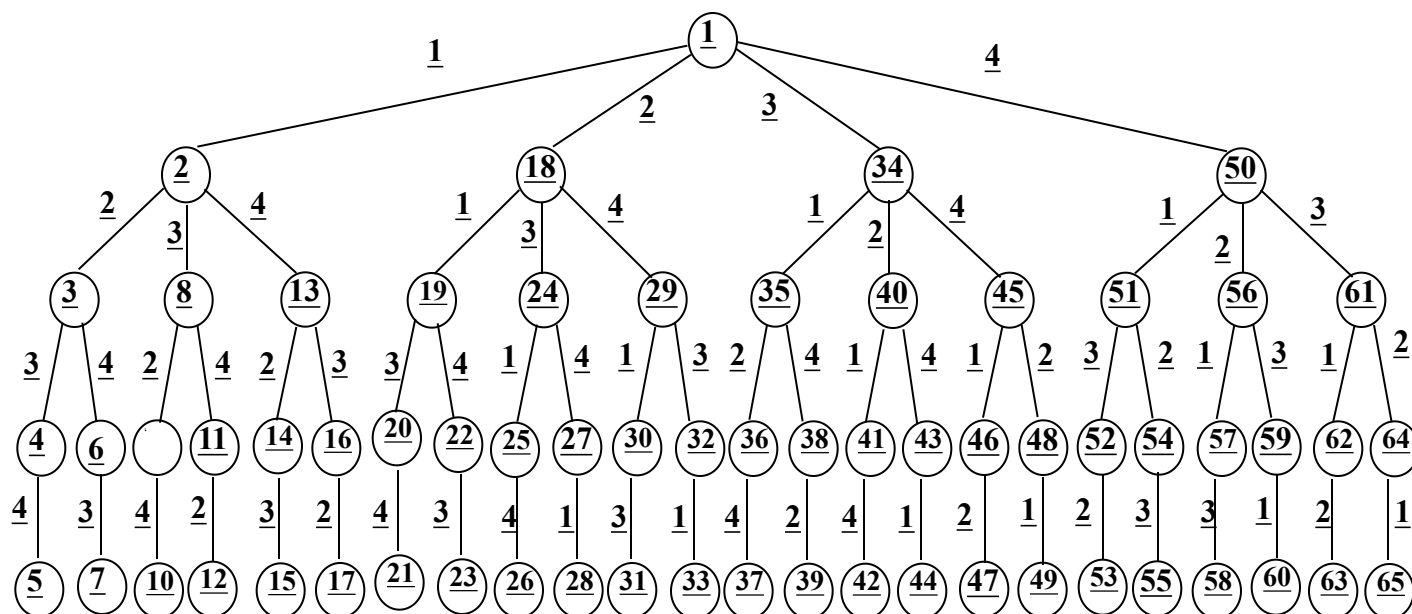
例：n后问题

- 确定问题状态：问题的状态即棋盘的布局状态。
- 构造状态空间树：状态空间树的根为空棋盘，每个布局的下一步可能布局是该布局结点的子结点。由于在每行中有且只有一个皇后，因此可采用逐行布局的方式（起到了剪枝的作用），即每个布局有 n 个子结点。



- 设4个皇后为 x_i ， 分别在第 i 行($i=1, 2, 3, 4$)；
- 问题的解状态： 可以用 $(1, x_1), (2, x_2), \dots, (4, x_4)$ 表示4个皇后的位置；
- 由于行号固定， 可简单记为： (x_1, x_2, x_3, x_4) ； 例如： $(4, 2, 1, 3)$
- 问题的解空间： $(x_1, x_2, x_3, x_4), 1 \leq x_i \leq 4 (i=1, 2, 3, 4)$ ， 共 $4!$ 个状态；

4皇后问题解空间的树结构



约束条件

- 任意两个皇后不能位于同一行上;
 - 任意两个皇后不能位于同一列上,
- 所以解向量 X 必须满足约束条件:

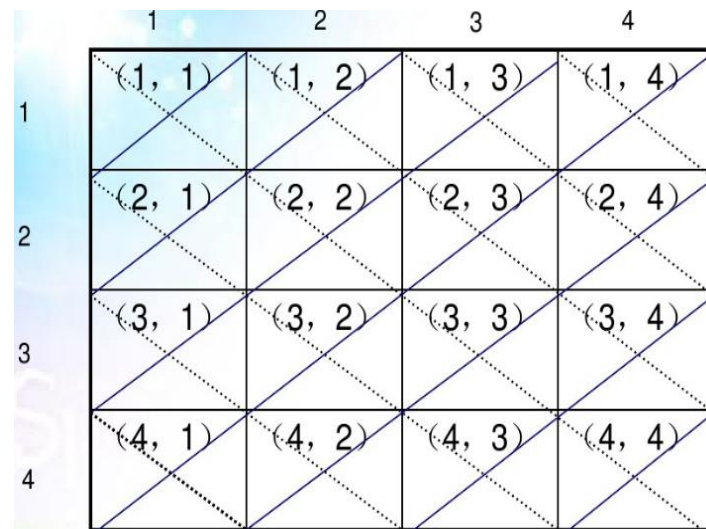
当 $i \neq j$ 时, $x_i \neq x_j$ (式1)

1		Q		
2				Q
3	Q			
4			Q	
	1	2	3	4

约束条件

- 任意两个皇后不能在同一条斜线上：若两个皇后摆放的位置分别是 (i, x_i) 和 (j, x_j) ，若在斜率为-1的斜线上，则有 $i-j = x_i - x_j$ ；若在斜率为1的斜线上，则有 $i+j = x_i + x_j$ 。综合两种情况，解向量 $\mathbf{X}$ 必须满足约束条件：

$$|i - x_i| \neq |j - x_j| \quad (\text{式2})$$



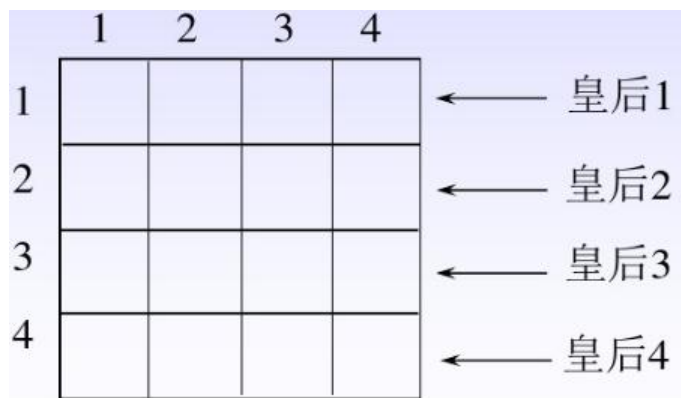
原问题即：在解空间中寻找符合约束条件的解状态。

搜索解空间，剪枝

- (1) 从空棋盘起，逐行放置棋子。
- (2) 每在一个布局中放下一个棋子，即推演到一个新的布局。
- (3) 如果当前行上没有可合法放置棋子的位置，则回溯到上一行，重新布放上一行的棋子。

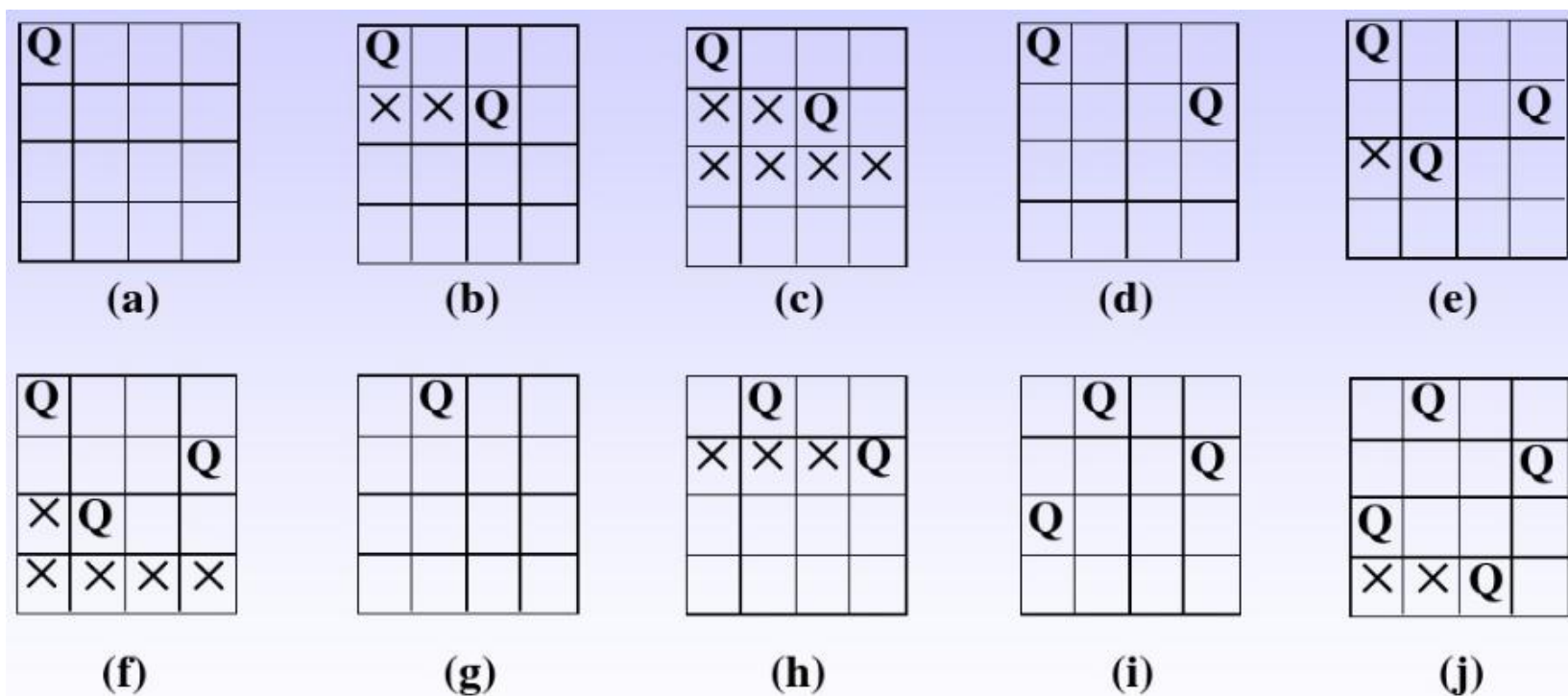
为了简化问题，下面讨论4皇后问题。

4皇后问题的解空间树是一个完全4叉树，树的根结点表示搜索的初始状态，从根结点到第2层结点对应皇后1在棋盘
中第1行可能摆放的位置，从第2层到第3层结点对应皇后2
在棋盘第2行的可能摆放的位置，以此类推。

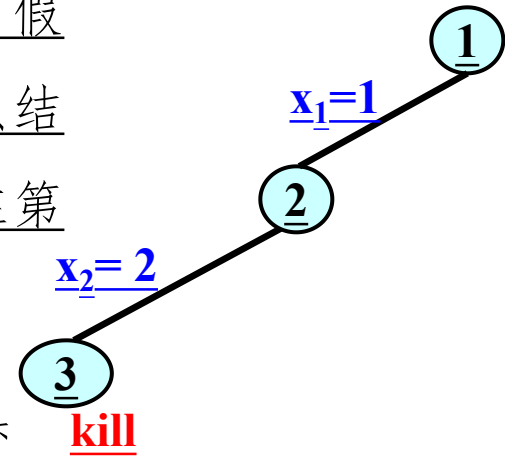


n后问题

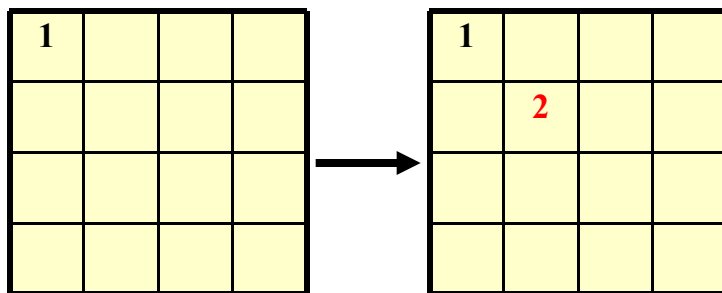
回溯法求解4皇后问题的搜索过程：



- 开始把根结点1作为唯一的活结点，根结点就成为E-结点而且路径为()；接着生成子结点，假设按自然数递增的次序来生成子结点，那么结点2被生成，这条路径为(1)，即把皇后1放在第1列上。

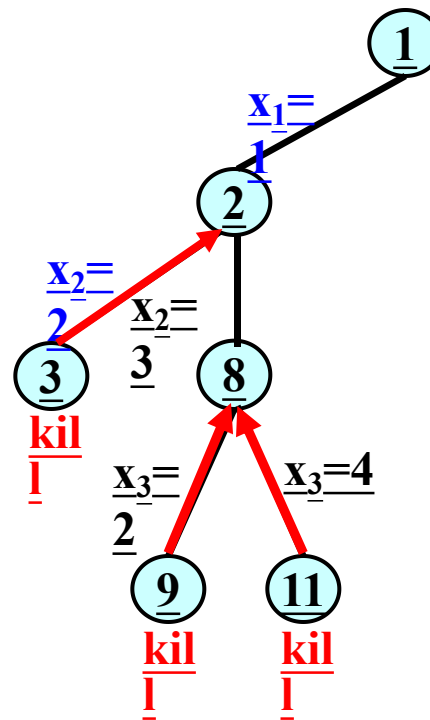
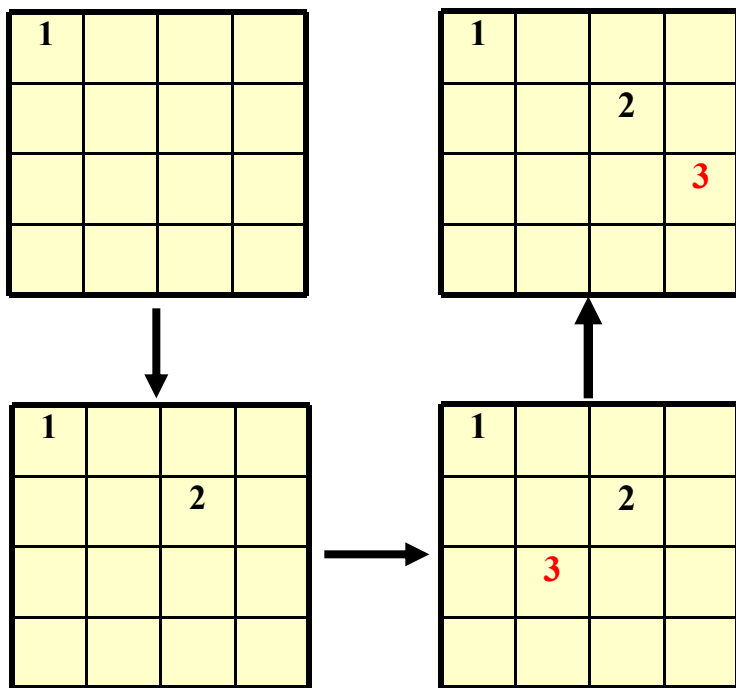


- 结点2变成E-结点，它再生成结点3，路径变为(1, 2)，即皇后1在第1列上，皇后2在第2列上，所以结点3被杀死，此时应回溯。



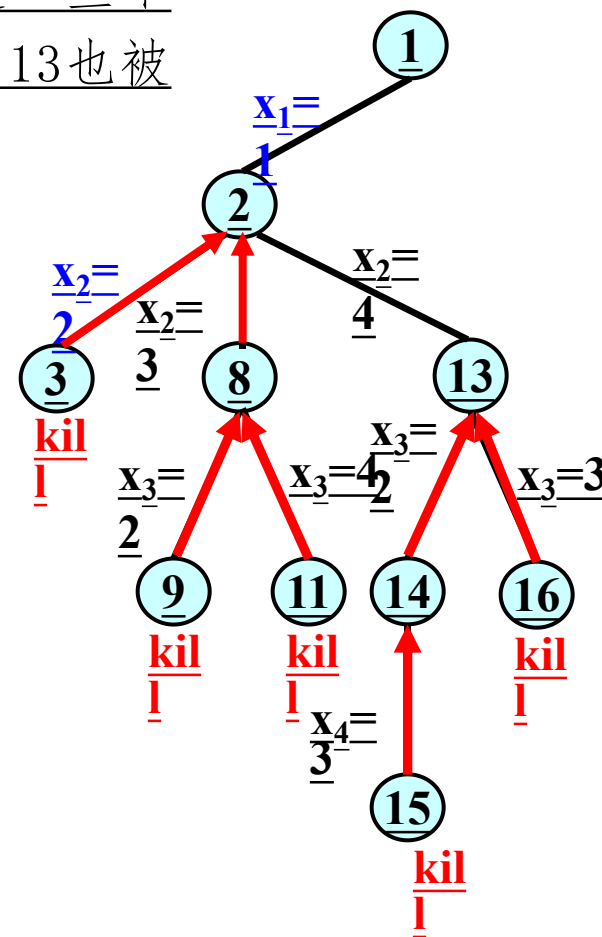
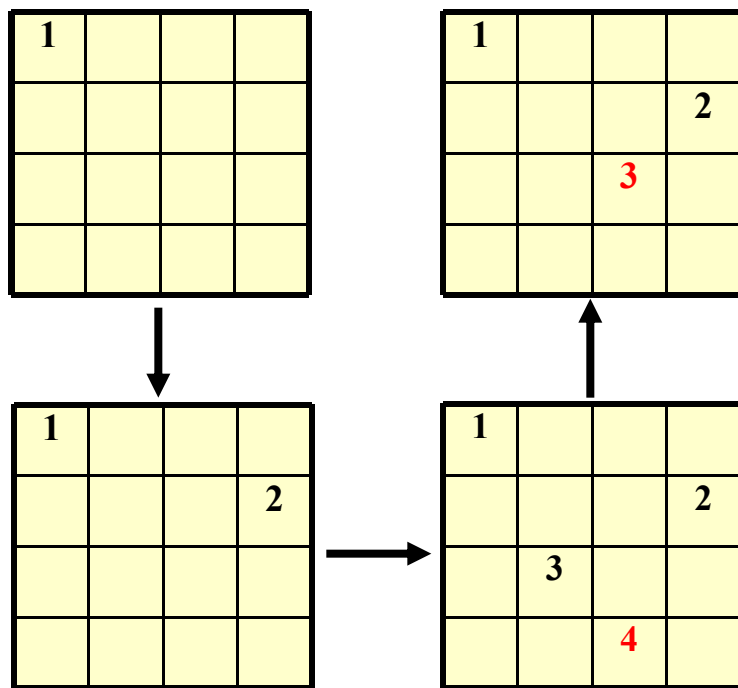
n后问题

- 回溯到结点2生成结点8, 路径变为(1, 3), 则结点8成为E-结点, 它生成结点9和结点11都会被杀死(即它的儿子表示不可能导致答案的棋盘格局), 所以结点8也被杀死, 应回溯。



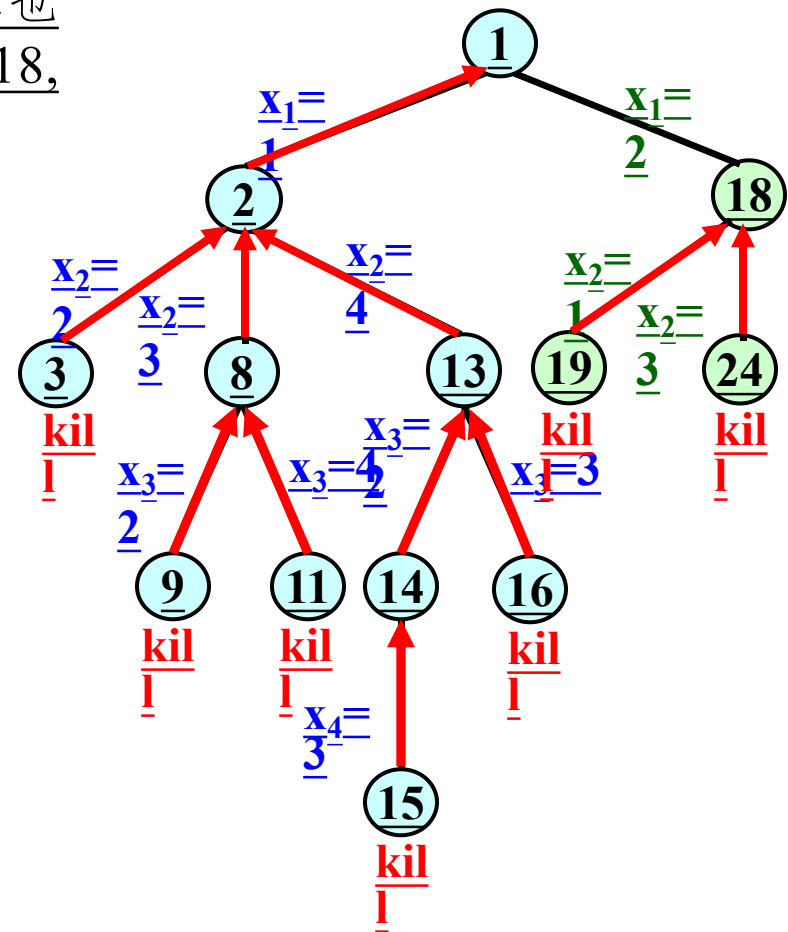
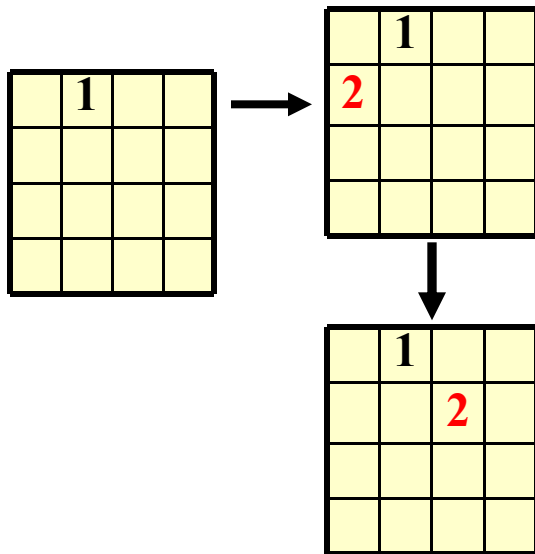
n后问题

- 回溯到结点2生成结点13，路径变为(1, 4)，结点13成为E-结点，由于它的儿子表示的是一些不可能导致答案结点的棋盘格局，因此结点13也被杀死，应回溯。

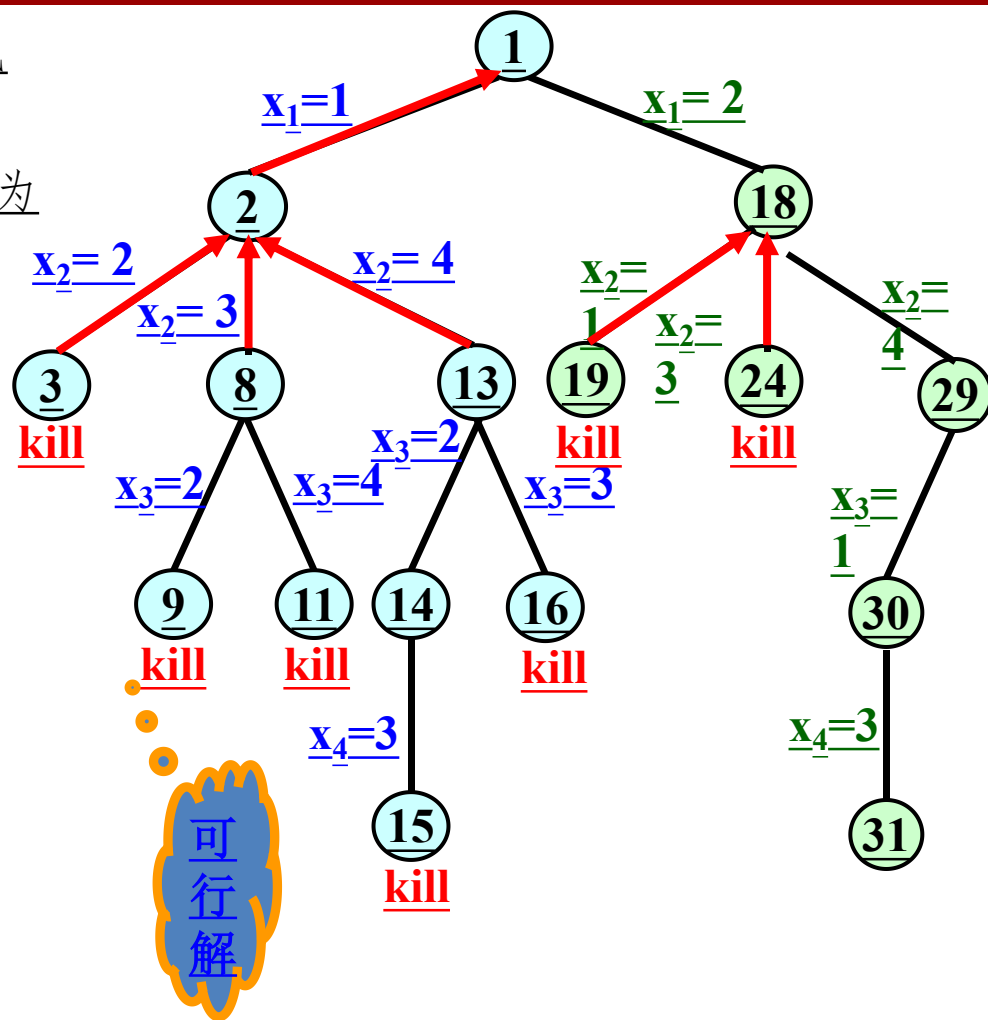
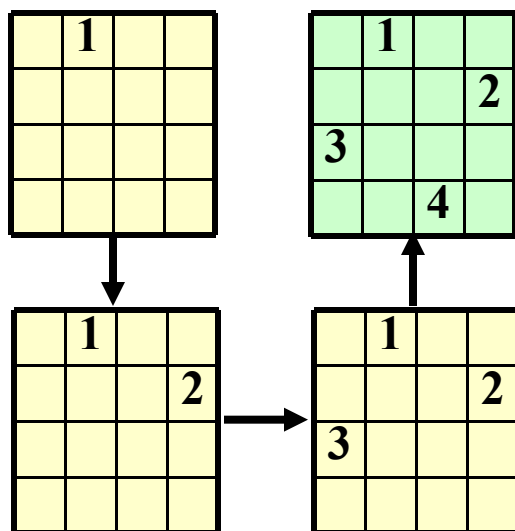


n后问题

- 结点2的所有儿子表示的都是不可能导致答案的棋盘格局, 因此结点2也被杀死; 再回溯到结点1生成结点18, 路径变为(2)。
- 结点18的子结点19、结点24被杀死, 应回溯。

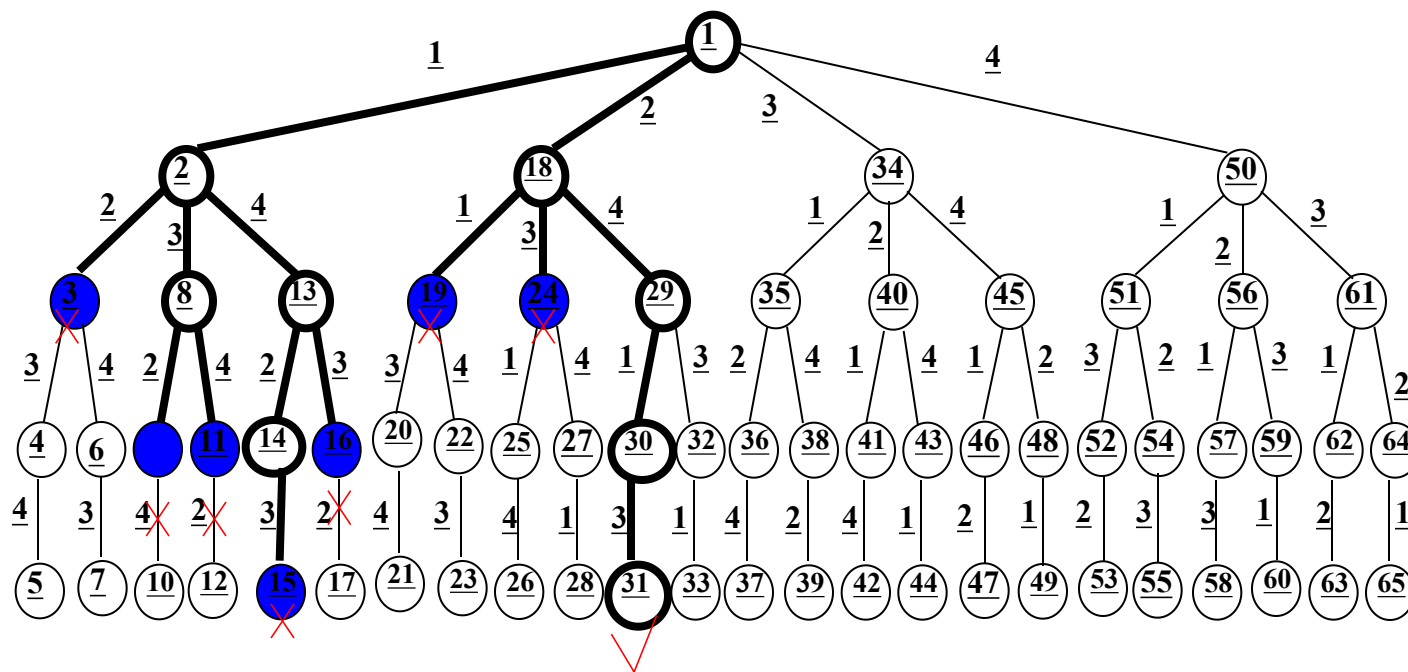


- 结点18生成结点29，结点29成为E-结点，路径变为(2,4)；
- 结点29生成结点30，路径变为(2,4,1)
- 结点30生成结点31，路径变为(2,4,1,3)，找到一个4-皇后问题的可行解



n后问题

n=4的n皇后问题的搜索、剪枝与回溯



图的 m 着色问题

图的着色问题是由地图的着色问题引申而来的：用 m 种颜色为地图着色，使得地图上的每一个区域着一种颜色，且相邻区域颜色不同。

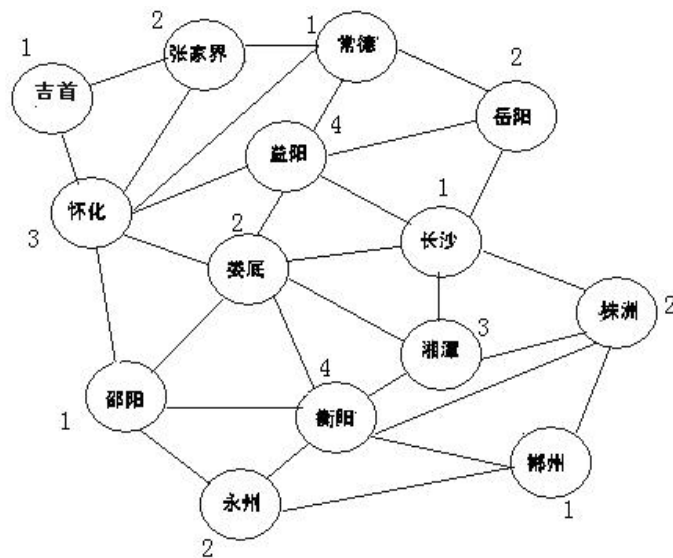


图的m着色问题

问题建模: 如果把每一个区域收缩为一个顶点, 把相邻两个区域用一条边相连接, 就可以把一个区域图抽象为一个平面图。



(a)

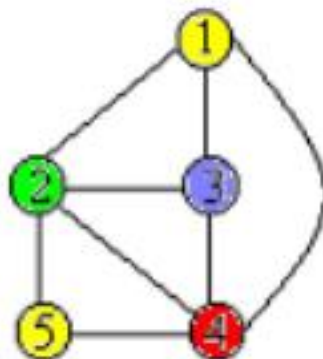
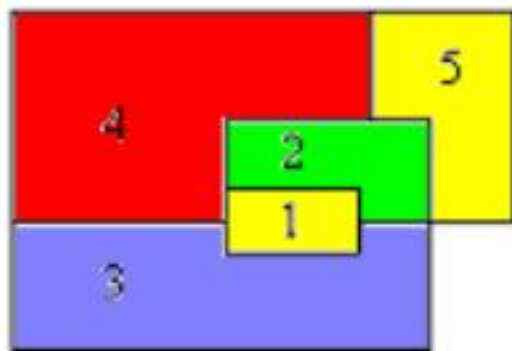


(b)

例如, 图 (a) 所示的区域图可抽象为 (b) 所表示的平面图。

图的 m 着色问题

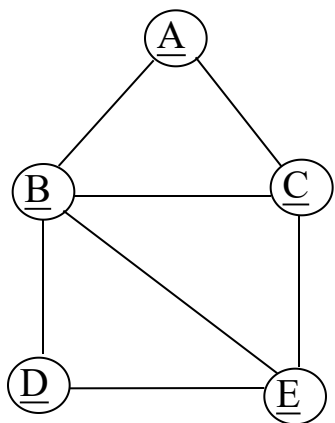
- 图着色问题描述为：给定无向连通图 $G=(V, E)$ 和正整数 m ，求最小的整数 m ，使得用 m 种颜色对 G 中的顶点着色，使得任意两个相邻顶点着色不同。



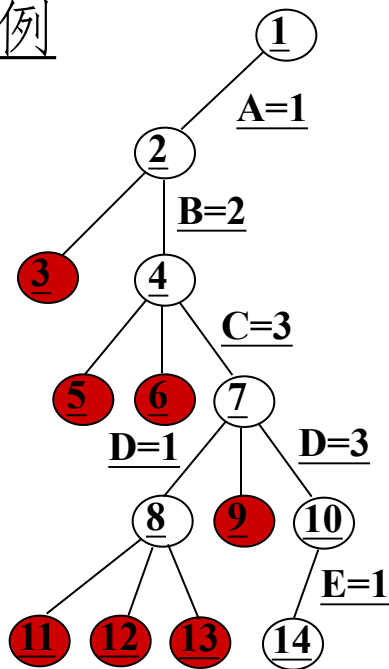
- 由于用 m 种颜色为无向图 $G=(V, E)$ 着色，其中， V 的顶点个数为 n ，可以用一个 n 元组 $C=(c_1, c_2, \dots, c_n)$ 来描述图的一种可能着色，其中， $c_i \in \{1, 2, \dots, m\} (1 \leq i \leq n)$ 表示赋予顶点 i 的颜色。
- 例如，5元组 $(1, 2, 2, 3, 1)$ 表示对具有5个顶点的无向图的一种着色，顶点1着颜色1，顶点2着颜色2，顶点3着颜色2，如此等等。
- 如果在 n 元组 C 中，所有相邻顶点都不会着相同颜色，就称此 n 元组为可行解，否则为无效解。

图的m着色问题

回溯法求解图着色问题示例



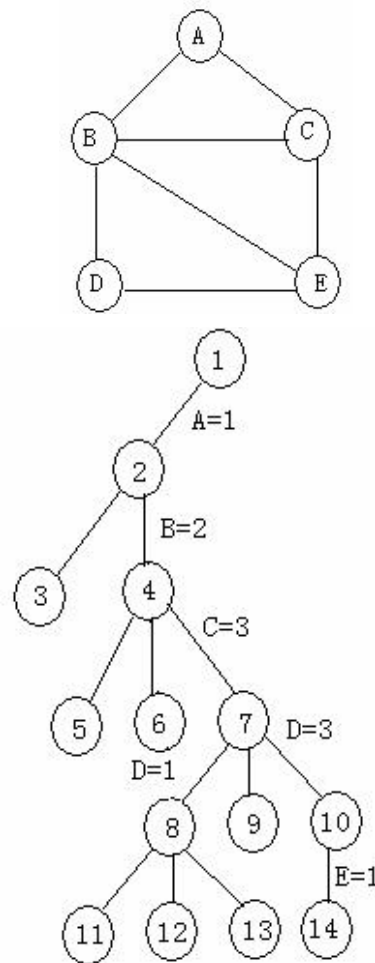
(a) 一个无向图



(b) 回溯法搜索空间

图的m着色问题

- ① 把5元组初始化为 $(0,0,0,0,0)$ ，从根结点开始向下搜索，以颜色1为顶点A着色，生成结点2时，产生 $(1,0,0,0,0)$ ，是个有效着色。
- ② 以颜色1为顶点B着色生成结点3时，产生 $(1,1,0,0,0)$ ，是个无效着色，结点3为d 结点。
- ③ 以颜色2为顶点B着色生成结点4，产生 $(1,2,0,0,0)$ ，是个有效着色。
- ④ 分别以颜色1和2为顶点C着色生成结点5和6，产生 $(1,2,1,0,0)$ 和 $(1,2,2,0,0)$ ，都是无效着色，因此结点5和6都是d 结点。
- ⑤ 以颜色3为顶点C着色，产生 $(1,2,3,0,0)$ ，是个有效着色。重复上述步骤，最后得到有效着色 $(1,2,3,3,1)$ 。



回溯法求解图着色问题：

- 首先把所有顶点的颜色初始化为0，然后依次为每个顶点着色。如果其中 i 个顶点已经着色，并且相邻两个顶点的颜色都不一样，就称当前的着色是有效的局部着色；否则，就称为无效的着色。
- 如果由根节点到当前节点路径上的着色，对应于一个有效着色，并且路径的长度小于 n ，那么相应的着色是有效的局部着色。这时，就从当前节点出发，继续探索它的儿子节点，并把儿子结点标记为当前结点。在另一方面，如果在相应路径上搜索不到有效的着色，就把当前结点标记为 $d_$ 结点，并把控制转移去搜索对应于另一种颜色的兄弟结点。
- 如果对所有 m 个兄弟结点，都搜索不到一种有效的着色，就回溯到它的父亲结点，并把父亲结点标记为 $d_$ 结点，转移去搜索父亲结点的兄弟结点。这种搜索过程一直进行，直到根结点变为 $d_$ 结点，或者搜索路径长度等于 n ，并找到了一个有效的着色为止。

设数组 $color[n]$ 表示顶点的着色情况，回溯法求解 m 着色问题的算法如下：

1. 将数组 $color[n]$ 初始化为0;
2. $k=1$;
3. while ($k \geq 1$)
 - 3.1 依次考察每一种颜色，若顶点 k 的着色与其他顶点的着色不发生冲突，则转步骤3.2；否则，搜索下一个颜色；
 - 3.2 若顶点已全部着色，则输出数组 $color[n]$ ，返回；
 - 3.3 否则，
 - 3.3.1 若顶点 k 是一个合法着色，则 $k=k+1$ ，转步骤3处理下一个顶点；
 - 3.3.2 否则，重置顶点 k 的着色情况， $k=k-1$ ，转步骤3。

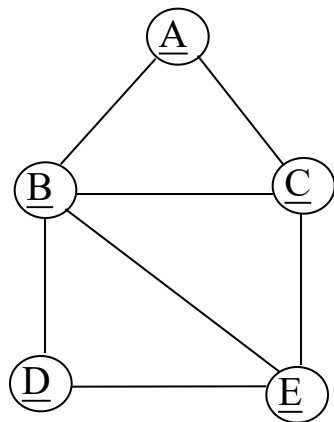
图的m着色问题

- 如何提升算法效率？

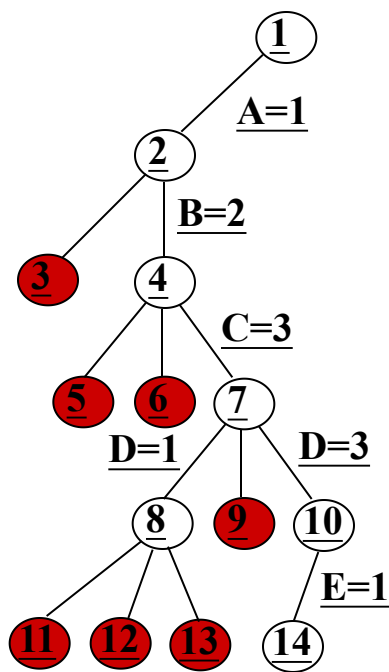
- ✓ 对称性？

- ✓ 变量的赋值顺序？

- ✓ ...



(a) 一个无向图



(b) 回溯法搜索空间

- 最坏情况下的时间 $W(n) = p(n) * f(n)$ ，其中 $p(n)$ 为每个结点时间， $f(n)$ 为结点个数
- 影响回溯算法效率的因素
 - 搜索树的结构
 - ✓ 分支情况：分支是否均匀
 - ✓ 树的深度
 - ✓ 对称程度：对称适合裁剪
 - 解的分布：
 - ✓ 在不同子树中分布是否均匀
 - ✓ 分布深度
 - 约束条件的判断：计算简单
- 改进途径：
 - 根据树分支设计优先策略：
 - ✓ 结点少的分支优先
，解多的分支优先
 - 利用搜索树的对称性剪裁子树

1. 以图的着色问题为例，如何进一步改进回溯法的性能？
思路：

- 变量的赋值顺序：下一步该给哪个变量赋值，按什么顺序给该变量赋值？
- 每步搜索应该做怎样的推理？当前变量的赋值会对其他未赋值变量产生什么约束，怎样利用这种约束以提高效率。
- 当遇到某个失败的变量赋值时，怎样避免同样的失败？就是说如何找到对这种失败起到关键作用的某个变量赋值。

- 设立**代价函数**
 - 函数值以该结点为根的搜索树中的所有可行解的目标函数值的上界
 - 父结点的代价不小于子结点的代价
- 设立**界**
 - 代表当时已经得到的可行解的目标函数的最大值
 - 界的设定初值可以设为 ∞
 - 可行解的目标函数值大于当时的界，进行更新
- 搜索中停止分支的依据
 - 不满足约束条件或者其代价函数小于当时的界

- first proposed by Ailsa Land and Alison Doig ^[1] in 1960
- A branch and bound algorithm operates according to two principles (from Wikipedia):
 - It recursively splits the search space into smaller spaces, then minimizing $f(x)$ on these smaller spaces; the splitting is called *branching*.
 - Branching alone would amount to **brute-force** enumeration of candidate solutions and testing them all. To improve on the performance of brute-force search, a B&B algorithm keeps track of *bounds* on the minimum that it is trying to find, and uses these bounds to "**prune**" the search space, eliminating candidate solutions that it can prove will not contain an optimal solution.

[1] A. H. Land and A. G. Doig (1960). "An automatic method of solving discrete programming problems". *Econometrica*. **28**(3). pp. 497–520. doi:10.2307/1910129. 67

1. 填字游戏：在 3×3 个方格的方阵中要填入数字1到 N （ $N \geq 10$ ）内的某9个数字，每个方格填一个整数，使得所有相邻两个方格内的两个整数之和为质数。试求出所有满足这个要求的各种数字填法。



算法设计与分析

讲授内容：回溯与分支限界(不作为考试范围)

2025年11月10日

- 约束满足问题简介
- 回溯搜索的改进技术（针对约束满足问题）
 - 变量和取值顺序
 - 通过约束传播信息
- 局部搜索
- 问题的结构

- 约束满足问题 (Constraint Satisfying Problem, CSP) :
 - 由一个变量集合 $\{X_1 \sim X_n\}$ 和一个约束集合 $\{C_1 \sim C_m\}$ 定义;
 - 每个变量都有一个非空可能值域 D_i ;
 - 每个约束指定了包含若干变量的一个子集内各变量的赋值范围。
- 例如: 地图染色问题, N-皇后问题。
- CSP的一个状态: 对一些或全部变量的赋值 $\{X_i=v_i, X_j=v_j, \dots\}$ 。

- 一个不违反任何约束的对变量的赋值称为**相容赋值或合法赋值**。
- 对每个变量都进行赋值称为**完全赋值**。
- 一个（一组）对变量的赋值，若既是相容赋值又是完全赋值，则这个（组）赋值是**CSP问题的解**。
 - 某些CSP问题要求问题的解能使目标函数最大化——**约束优化**。
- CSP问题常常可以可视化，表示为**约束图**，更直观地显示问题，帮助思考问题的答案。

- 根据变量的类型划分：离散值域和连续值域。
- 变量——离散值域
 - 有限值域，如地图染色问题，八皇后问题。
 - 无限值域，如整数集合或者字符串集合。
 - 例如，对于作业规划问题，无法枚举所有可能取值，要使用约束语言(线性约束/非线性约束)描述，如 $\text{StartJob}_1 + 5 \leq \text{StratJob}_3$ 。
- 变量——连续值域
- 最著名的连续值域CSP是线性规划问题。
 - 线性规划中的约束必须是构成一个凸多边形的一组线性不等式。
 - 线性规划问题可以在变量个数的多项式时间内求解。

- 根据约束的类型划分：
 - 线性或非线性约束。
 - 一元或多元约束。
 - 一元约束：只限制一个变量的取值
 - 二元约束与2个变量相关
 - 高阶约束：涉及3个或更多变量。
 - 通过引入辅助变量，转为二元约束。
 - 绝对约束 vs 偏好约束。
 - 我们仅讨论绝对约束。

- CSP问题的求解目标是找到相容的完全赋值，最朴素的想法是依次取变量的赋值组合并检查其是否满足约束条件。
 - 若CSP问题的任何一个变量的最大值域为 d ，那么可能的完全赋值数量为 $O(d^n)$ 。
 - 指数级计算量。

- 约束满足问题简介
- 回溯搜索的改进技术（针对约束满足问题）
 - 变量和取值顺序
 - 通过约束传播信息
- 局部搜索
- 问题的结构

例1、澳大利亚地图染色问题(1)

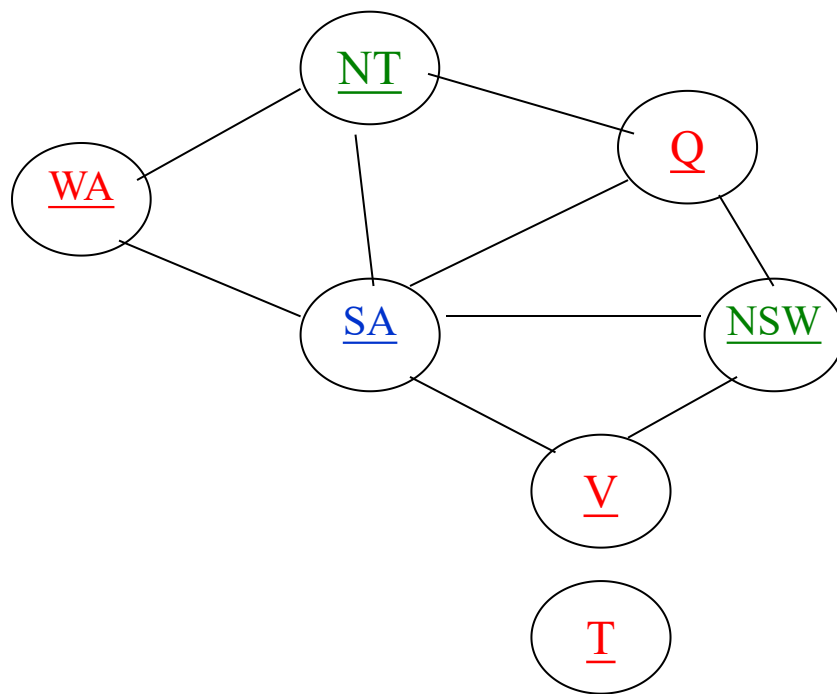
- 澳大利亚地图染色问题：用红绿蓝3色标出各省，相邻者颜色不同。



例1、澳大利亚地图染色问题(2)

- 对应于澳大利亚地图的约束图，相互关联的节点用边连接。

- 西澳大利亚 – WA
- 北领地 – NT
- 南澳大利亚 – SA
- 昆士兰 – Q
- 新南威尔士 – NSW
- 维多利亚 – V
- 塔斯马尼亚 – T

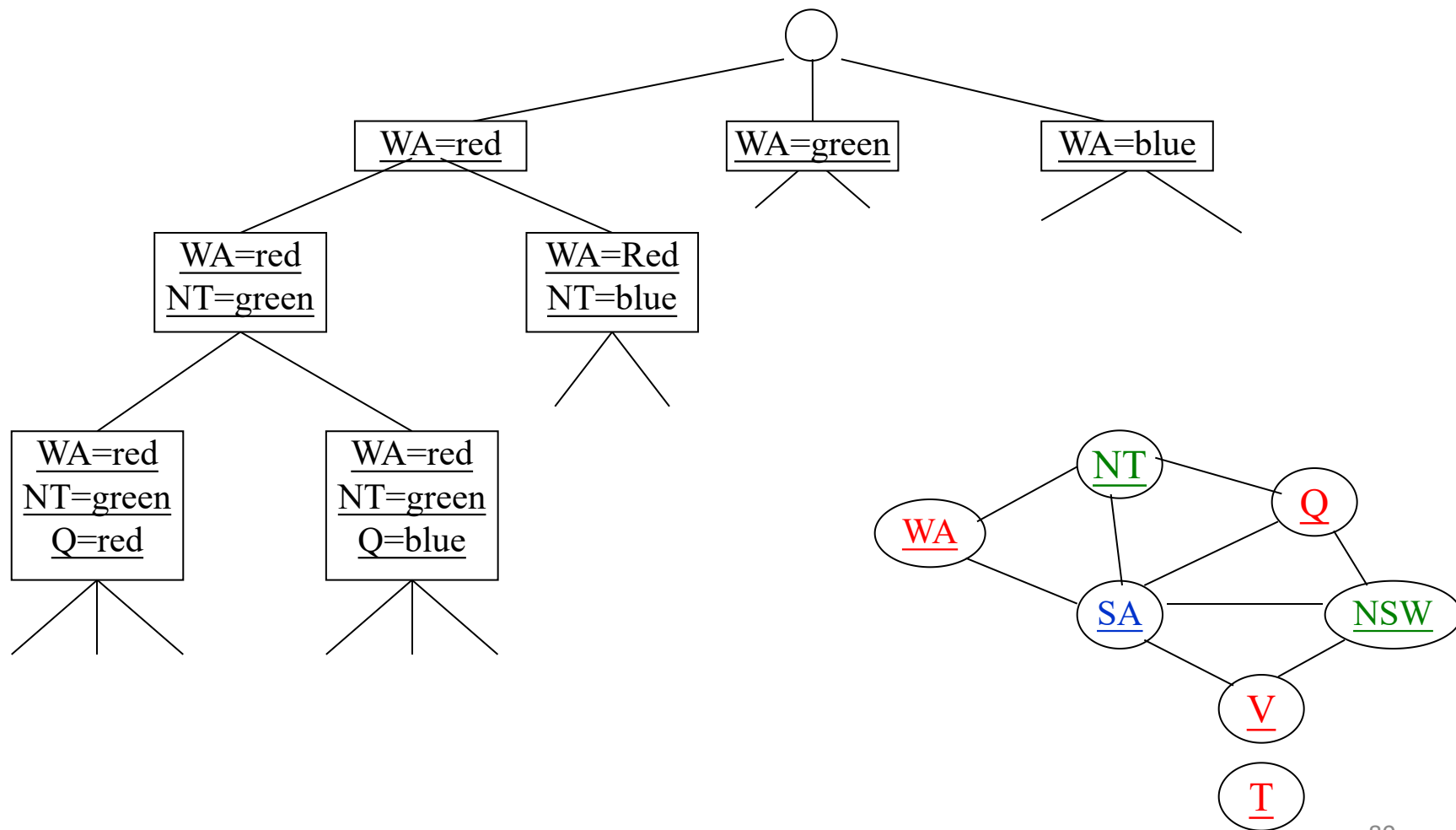


- 一组满足约束的完全赋值： { WA=R, NT=G, Q=R, SA=B, NSW=G, V=R, T=R }

- CSP问题具有一个性质：**可交换性**，变量赋值的顺序对结果没有影响。
 - 所有CSP搜索算法生成后继节点时，在搜索树每个节点上只考虑**单个变量**的可能赋值。
- CSP问题的求解：**深度优先的回溯搜索**。
 - 每次给一个变量赋值，当没有合法赋值（不满足约束时）就要推翻前一个变量的赋值，重新给其赋值，这就是回溯。

简单回溯法生成的搜索树

- 澳大利亚地图染色问题的搜索树



一个简单的回溯算法（深度优先）

1. **function** BackTracking-Search(csp) **returns** a solution, or failure
2. **return** Recursive-BackTracking({}, csp)

1. **function** Recursive-BackTracking(assignment, csp) **returns** a solution, or failure
2. **if** assignment is complete **then return** assignment
3. var ← **Select-Unassigned-Variable**(Variables[csp], assignment, csp)
4. **for** each value in Order-Domain-Values(var, assignment, csp) **do**
5. **if** value is consistent with assignment according to
 Constraints[csp] **then**
6. add { var=value } to assignment
7. result ← **Recursive-BackTracking**(assignment, csp)
8. **if** result != failure **then return** result
9. remove { var=value } from assignment
10. **return** failure

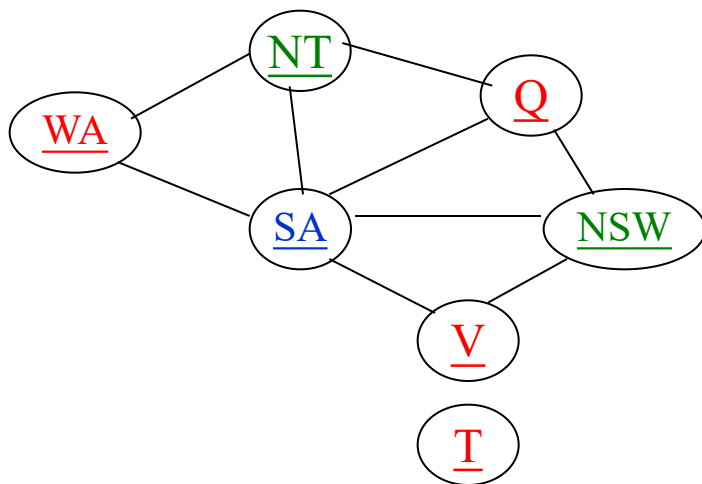
- 需改善无信息回溯搜索算法的性能。
- 通用改进方法的思路：
 - 下一步该给哪个变量赋值，按什么顺序给该变量赋值？
 - 每步搜索应该做怎样的推理？当前变量的赋值会对其他未赋值变量产生什么约束，怎样利用这种约束以提高效率。
 - 当遇到某个失败的变量赋值时，怎样避免同样的失败？就是说如何找到对这种失败起到关键作用的某个变量赋值。

- 基于变量和赋值次序的启发式
- 搜索与推理交错进行
- 智能回溯：向后看

- 基于变量和赋值次序的启发式
 - MRV（最少剩余值）启发式
 - 最少约束值启发式
 - 度启发式
- 搜索与推理交错进行
- 智能回溯：向后看

(1) 最少剩余值 (MRV) 启发式

- 随机的变量赋值排序难以产生高效率的搜索。
- **例如：**在WA=red/NT=green条件下选取SA赋值比Q要减少赋值次数(1:2)，并且一旦给定SA赋值以后，Q、NSW和V的赋值只有一个选择。

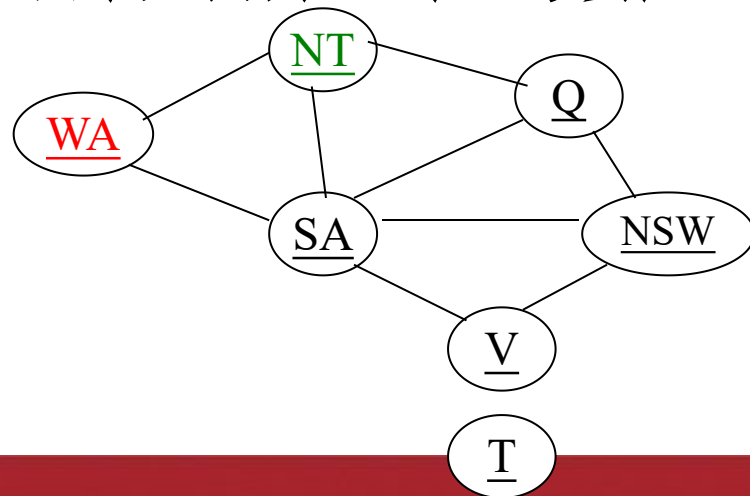


- 因此，应当选择合法取值最少的变量，即最少剩余值(MRV)启发式，也称为最受约束变量启发式或失败优先启发式。

—称为失败优先启发式是因为它可以很快找到失败的变量，从而引起搜索的剪枝，避免更多导致同样失败的搜索。

(2) 最少约束值启发式

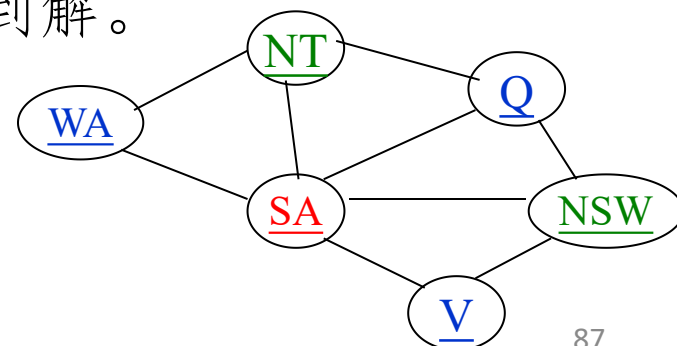
- **MRV启发式**：当有多个变量需要选择时，优先选择在当前约束下取值最少的变量。
 - **问题**：一旦变量被选定，算法就要决定它的取值的次序。
- **最少约束值启发式**：当赋值的变量有多个值选择时，优先的值应是约束图中排除邻居变量的可选值最少的，即优先选择为剩余变量的赋值留下最多选择的赋值。
 - **例如**，WA=red且NT=green时，如果给Q赋值，可以为blue或red，而Q=blue的选择不好，因为此时SA没有一个可选择的了。



(3) 度启发式

- **MRV启发式**：当有多个变量需要选择时，优先选择在当前约束下取值最少的变量。
 - **问题**：对澳大利亚地图着色问题中选择第一个染色问题根本没有帮助，因为初始的时候每个区域都有3种选择。
- **度启发式**：选择涉及对其他未赋值变量的约束数量大（与其他变量关联最多）的变量。
 - 地图染色例子中，度(SA)=5，其他均为2或3。
 - 实际上，一旦选择了SA作为初始节点，应用度启发式求解本问题，则可以不经任何回溯就找到解。

SA=red $\Rightarrow$ NT=green $\Rightarrow$
Q=blue $\Rightarrow$ NSW=green $\Rightarrow$
WA=blue $\Rightarrow$ V=blue $\Rightarrow$ T=red



- 基于变量和赋值次序的启发式
- 搜索与推理交错进行
- 智能回溯：向后看

- 一般的回溯算法：只有在选择了一个变量的时候才考虑该变量的约束。
- 变量约束的启发式：在搜索的早些时候，或开始之前就考虑某些约束，从而降低搜索空间。
 - ① 约束传播
 - ② 最简单的推理形式：前向检验

(1) 前向检验

- **前向检验**：如果X被赋值，检查与X相连的那些变量Y，看看它们是否满足相关约束，并从Y的值域中删去与X取值矛盾的那些赋值。

蓝色字体为赋值结果

WA=red

Q=green

V=blue

WA	NT	Q	NSW	V	SA
RGB	RGB	RGB	RGB	RGB	RGB
R	GB	RGB	RGB	RGB	GB
R	B	G	R B	RGB	B
R	B	G	R	B	---

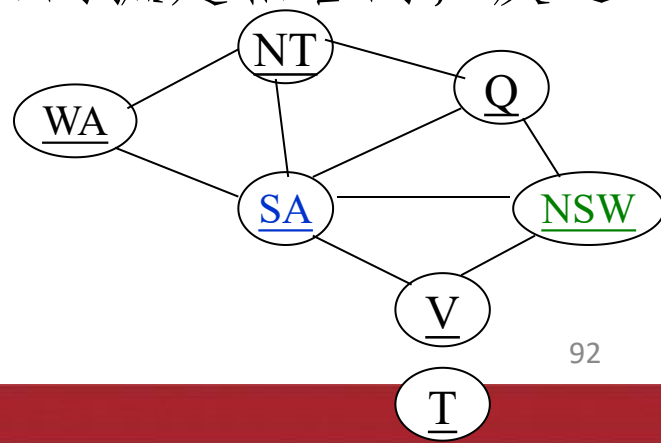
- 赋值V=blue引起矛盾，此时SA赋值为空，不满足问题约束，此时算法就要立刻回溯。

(1) 前向检验

- 前向检验可与MRV启发式相结合。
 - 实际上，MRV要做的就是向前找合适的变量。
- 注意：这里只是检验一步，即和当前节点是否矛盾。
 - 前向检验看得不够远。虽然前向检验能检验出许多矛盾，它还是不能检验出所有的矛盾。
 - 被检验节点之间的约束检验还不能进行。
 - 改进：约束传播。

(2) 约束传播

- **约束传播**：使用约束来减小一个变量的合法取值范围，从而影响到与此变量有约束关系的另一变量的取值。
- **弧相容**：依次检验约束图中各个相关节点对。
注意，弧是有向弧。
 - 例如，给定SA/NSW当前值域，如果对于SA的每个取值 x ，NSW都有某个 y 和 x 相容，则SA到NSW的弧是相容的；反过来是NSW到SA的弧相容。



约束传播：弧相容

- 在地图染色约束的赋值过程中：第4行SA={blue}, NSW={red, blue}, 则SA的取值有一个NSW=red与之相容；反过来NSW=blue, 则SA为空值, 即不相容, 通过删除NSW值域中的blue可使其相容。
- 弧相容检测也能较早发现矛盾, 如第4行SA和NT值域均为{blue}, 则必须删去SA=blue, 其值域变为空。
— 用弧相容能够更早地检测到矛盾。

蓝色字体为赋值结果

WA=red

Q=green

V=blue

WA	NT	Q	NSW	V	SA
RGB	RGB	RGB	RGB	RGB	RGB
R	GB	RGB	RGB	RGB	GB
R	B	G	R B	RGB	B
.....					93

1. 用队列记录需要检验不相容的弧。
 2. 每条弧 $[X_i, X_j]$ 依次从队列中删除并被检验。
 - 如果 X_i 值域中的任何一个值需要删除，则每个指向 X_i 的弧 $[X_k, X_i]$ 都必须重新插入队列进行检验。
 - 因为指向这个变量的弧可能产生新的不相容（原来可能就是因为这个值产生了它们之间的相容）。
-
- **时间复杂度：**二元CSP约束至多有 $O(n^2)$ 条弧；每条弧至多插入队列 d 次（ d 个取值），检验一条弧为 $O(d^2)$ ，因此算法的最坏情况下为 $O(n^2d^3)$ 。

1. function **AC-3**(csp) returns the CSP, possibly with reduced domains
2. inputs: csp, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$
3. local variables: queue, a queue of arcs, initially all the arcs in csp
4. while queue is not empty do
5. $(X_i, X_j) \leftarrow \text{Remove-First}(\text{queue})$
6. if **RM-Inconsistent-Values**(X_i, X_j) then
7. for each X_k in $\text{Neighbors}[X_i]$ do
8. add (X_k, X_i) to queue

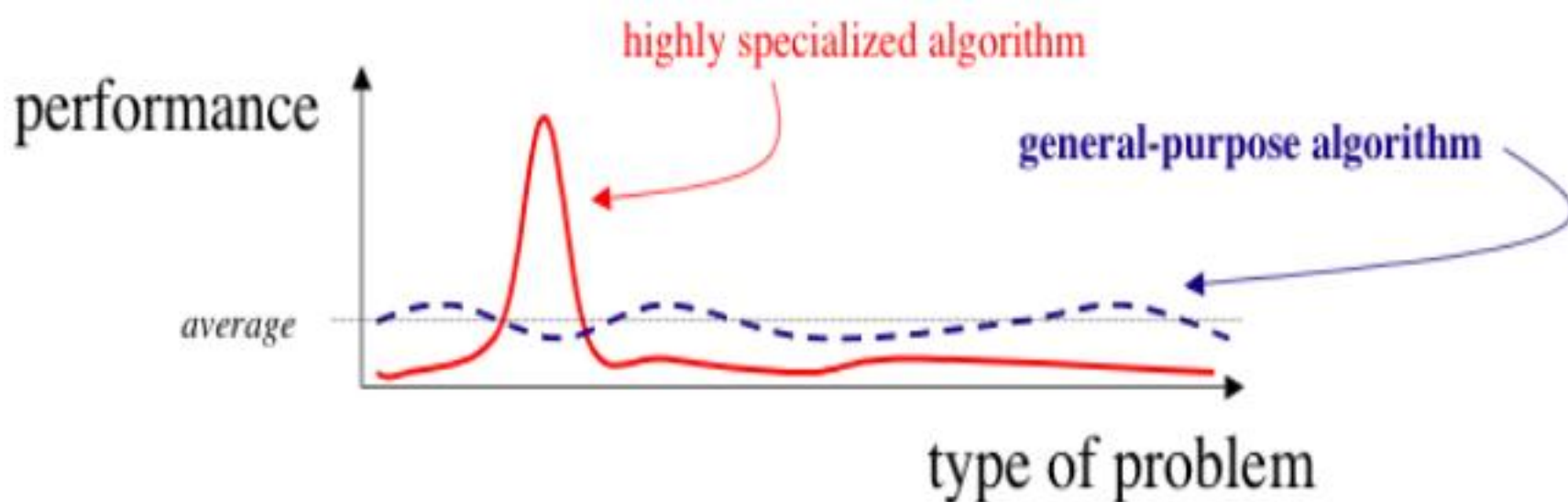
1. function **RM-Inconsistent-Values**(X_i, X_j)
returns true iff remove a value
1. removed $\leftarrow$ false
2. for each x in $\text{Domain}[X_i]$ do
3. if no value y in $\text{Domain}[X_j]$ allows (x, y) to satisfy constraint(X_i, X_j)
4. then delete x from $\text{Domain}[X_i]$; removed $\leftarrow$ true
5. return removed

- 弧相容检验可以用作开始搜索之前的预处理。
- 弧相容检验也可以在搜索过程中每次赋值后用作一个约束传播步骤。
 - 即反复检测某个变量值域中的不相容弧，进行值删除，直到不再有矛盾。
 - 称为保持弧相容（MAC）。

从弧相容推广到 k 相容

- 如果对于任何 $k-1$ 个变量的相容赋值，第 k 个变量总能被赋予一个与前 $k-1$ 个变量相容的值，那么该CSP问题是 k 相容的。
- 1相容是指每个单独的变量自己是不矛盾的，也称为节点相容。
- 弧相容=2相容。
- 3相容是指任何一对相邻的变量总可以扩展到第三个邻居变量，也称为路径相容。
- 如果一个图是 k 相容的，也是 $k-1$ 相容的、 $k-2$ 相容的，……，直到1相容，那么这个图是强 k 相容的。

- 如果一个CSP问题有 n 个结点，而且它是强 n 相容的，则不需要回溯就能求解这个问题。
 - 可以在 $O(nd)$ 时间内找到解。
- **No free Lunch:** 任何建立 n 相容的算法在最坏情况下必须花费 n 的指数级时间。
- **从弧相容到 n 相容:** 执行较强的相容性检验会花费更多的时间，但是会更有效地降低分支因子和检测出矛盾的不完全赋值。



- 实际问题中出现的**特殊约束**，用**专用算法**处理其效率要比**通用的约束处理算法**高很多。

- 例如，**AllDiff约束**，**变量取值各不相同**
 - 假设约束涉及 m 个变量，所有变量共有 n 个取值，如果 $m > n$ ，则此约束不能被满足。
- 引出相应的**算法**:
 - 删除约束中只有单值值域的变量，将这些变量的取值从其余变量值域中删去；
 - 对单值变量重复此过程；
 - 如果得到空的值域、或剩下的变量数大于取值数，则产生矛盾。

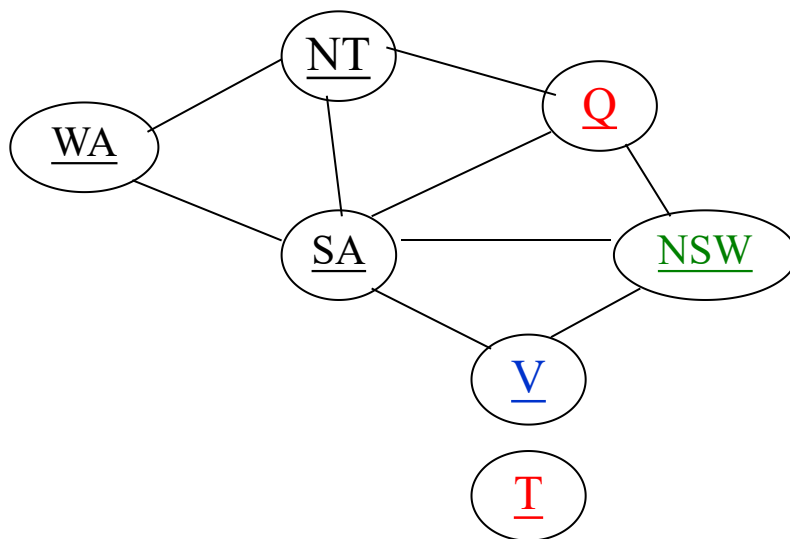
- 资源约束，有时称为 **atmost** 约束。
- 例如，用 $PA_1, \dots, PA_4$ 表示分配给四项任务的人员个数，人员数总计不超过10人的约束记为 $\text{atmost}(10, PA_1, \dots, PA_4)$ 。
 - 如果每个变量的赋值为 $\{3, 4, 5, 6\}$ ，就不能满足 **atmost** 约束。
 - 通过检验当前值域中的最小值之和就能检测出矛盾。

- 对于大型的整数值的资源限制问题，用整数集合来表示每个变量的值域然后通过相容性检验方法逐步削减集合，通常是不可能的。
 - 取代办法：值域用上界和下界来表示，并通过边界传播来管理。
- 例、假设有2次航班，271和272，它们分别有165和385个座位。每次航班可承载的初始值域为 $\text{Flight271} \in [0, 165]$, $\text{Flight272} \in [0, 385]$ 。
设又增加一个约束，两次航班所承载的总乘客数必须是420。通过边界传播约束，可以把这两个值域削减到 $\text{Flight271} \in [35, 165]$, $\text{Flight272} \in [255, 385]$ 。
- 如果对于每个变量X和它取值的上下界，每个变量Y都存在某个取值满足X和Y之间的约束，我们称该CSP问题是边界相容的。

- 基于变量和赋值次序的启发式
- 搜索与推理交错进行
- 智能回溯：向后看

- 在回溯算法中，当发现不满足约束即搜索失败时，则回到上一个变量并尝试下一个取值，这称为**历时回溯**。
 - 在很多情况下这样做是效率很低的，因为问题并不决定于上一个（甚至几个）变量的取值。
- 回溯应倒退到**导致失败的变量的集合**中的一个变量。
 - 该集合称为**冲突集**。
- **变量X的冲突集**是通过约束与X相连接的、先前已赋值变量的集合。

- 对于地图染色问题，设有不完全赋值
 $\{Q=\text{red}, \text{NSW}=\text{green}, V=\text{blue}, T=\text{red}\}$ 。



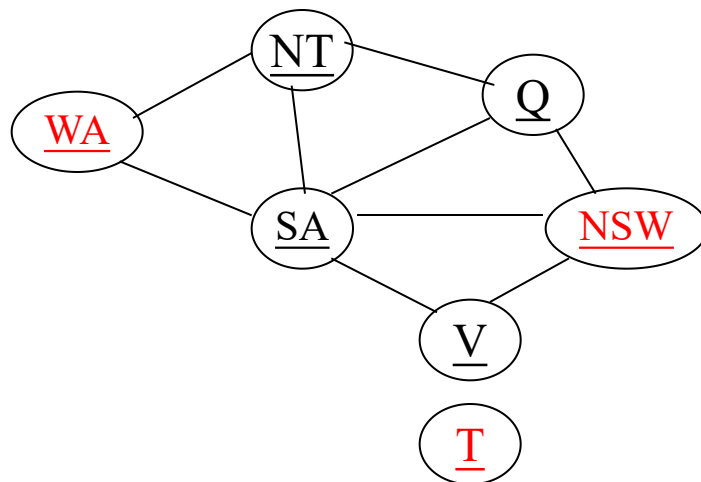
- 此时，给SA赋值将发现无法满足任何约束，SA的冲突集= $\{Q, \text{NSW}, V\}$ 。

- 后向跳转：回溯到冲突集中时间最近（最后赋值）的变量。
- 对于前向检验算法，可以很容易得到冲突集。
 - 基于X赋值的前向检验从变量Y的值域中删除一个值时，说明X和Y存在冲突，则显然X是Y的冲突集中的一个变量。
 - 当到达Y时，可知回溯到哪个变量。
- 对于弧相容检验，因为都是做取值相容的检测，只要在弧相容检验时增加一个变量集合记录即可。

- **问题：**后向跳转只出现在值域中的每个值都和当前的赋值有冲突的情况下，但是前向检验能检测到这个事件并且一旦到达这样的结点就阻止搜索。
 - **可以证明：**每个被后向跳转剪枝的分支在前向检验算法中也被剪枝。
 - 因此，简单的后向跳转在前向检验搜索中，或者在诸如MAC（保持弧相容）这样使用更强的相容性检验的搜索中是多余的。

冲突指导的后向跳转

- 很多情况下，一个分支发生很久以前就已经注定要失败了。
- 例，考虑不完全赋值 $\{WA=red, NSW=red\}$ 。
 - 假设下一个赋值尝试 $T=red$ ，然后给 NT, Q, V, SA 赋值。



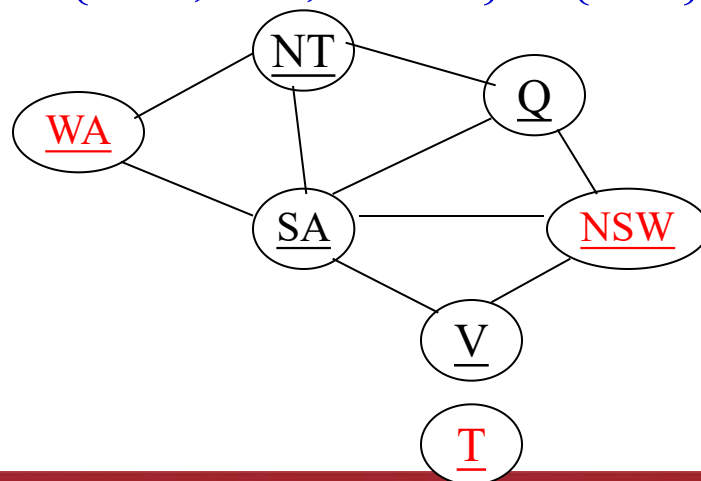
– 因为最后的四个变量 NT, Q, V, SA 没有相容的赋值，因此最终我们尝试了 NT 的所有可能取值。

– 现在的问题是向哪里回溯？

– 后向跳转行不通： NT 确实有和已赋值变量相容的值，但 NT 没有完整的由前面能导致失败的变量组成的冲突集。

- 变量冲突集更一般的情况：前面的变量集合使得当前变量连同任何后继变量一起没有相容解。
- 冲突指导的后向跳转处理：
 - 令 X_j 是当前变量， $\text{conf}(X_j)$ 是其冲突集。
 - 如果 X_j 每个可能取值都失败了，则后向跳转到 $\text{conf}(X_j)$ 中最近的一个变量 X_i ，令
$$\text{conf}(X_i) = \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_i\}。$$
 - 从 X_i 向前是无解的，从 X_i 回到某个以前的变量赋值。

- 例，考虑不完全赋值{WA=red, NSW=red}。
 - 假设下一个赋值尝试T=red，然后给NT, Q, V, SA赋值。
 - 因为最后四个变量NT, Q, V, SA没有相容的赋值，回溯。
 - SA的冲突集是{WA, NT, Q}，后向跳转到Q。
 - Q将SA的冲突集（减去Q）吸收到自己的冲突集里，则Q的新的冲突集为{WA, NT, NSW}。
 - 给定了{WA, NT, NSW}的赋值后，从Q向前是无解的。
 - 回溯到NT，NT的冲突集变为{WA, NT, NSW} - {NT} + {WA} = {WA, NSW}。
 - 后向跳转到NSW。

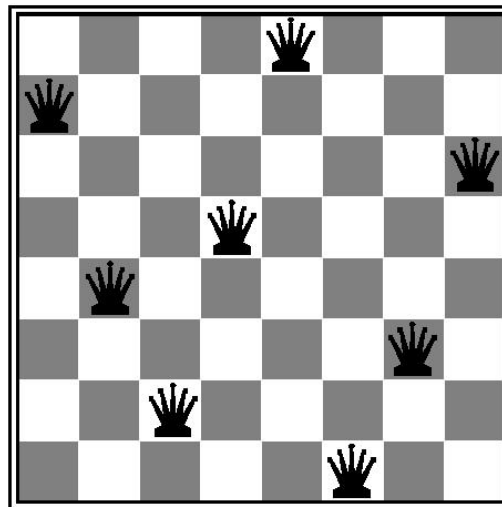
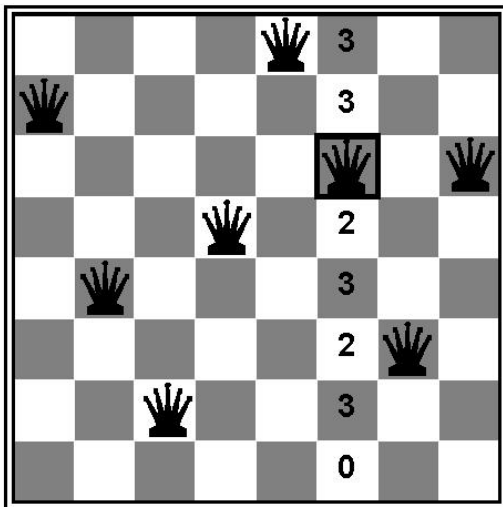
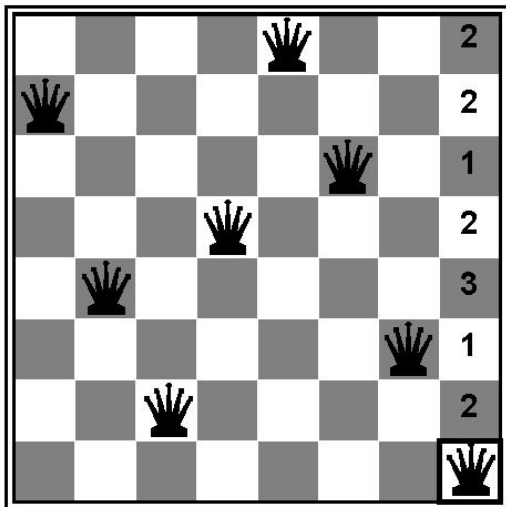


- 约束满足问题简介
- 回溯搜索的改进技术（针对约束满足问题）
 - 变量和取值顺序
 - 通过约束传播信息
- 局部搜索
- 问题的结构

- **完全状态的形式化**：初始状态给每个变量赋值，后继函数一次**改变**一个变量的取值。
- 在为变量选择新值时，最明显的启发式是：
 - 选择与其它变量的冲突最小的值，**最小冲突启发式**。

例、八皇后问题

- 用最小冲突算法解决八皇后问题。
 - 每一步选择一个皇后，在它所在的列中重新分配位置。
 - 算法将皇后移到最小冲突的方格中，最小冲突值有多个方格时则随机地选取一个。



约束满足问题的局部搜索

- 一个用局部搜索解决CSP问题的Min-conflicts算法。
- 1. function MIN-CONFLICTS(*csp*, *max_steps*) return solution or failure
- 2. inputs: *csp*, a constraint satisfaction problem
- 3. *max_steps*, the number of steps allowed before giving up
- 4. *current* $\leftarrow$ an initial complete assignment for *csp*
- 5. for *i* = 1 to *max_steps* do
- 6. if *current* is a solution for *csp* then return *current*
- 7. *var* $\leftarrow$ a randomly chosen, conflicted variable from
- 8. VARIABLES[*csp*]
- 9. *value* $\leftarrow$ the value *v* for *var* that minimizes
- 10. CONFLICTS(*var*, *v*, *current*, *csp*)
- 11. set *var* = *value* in *current*
- 12. return failure

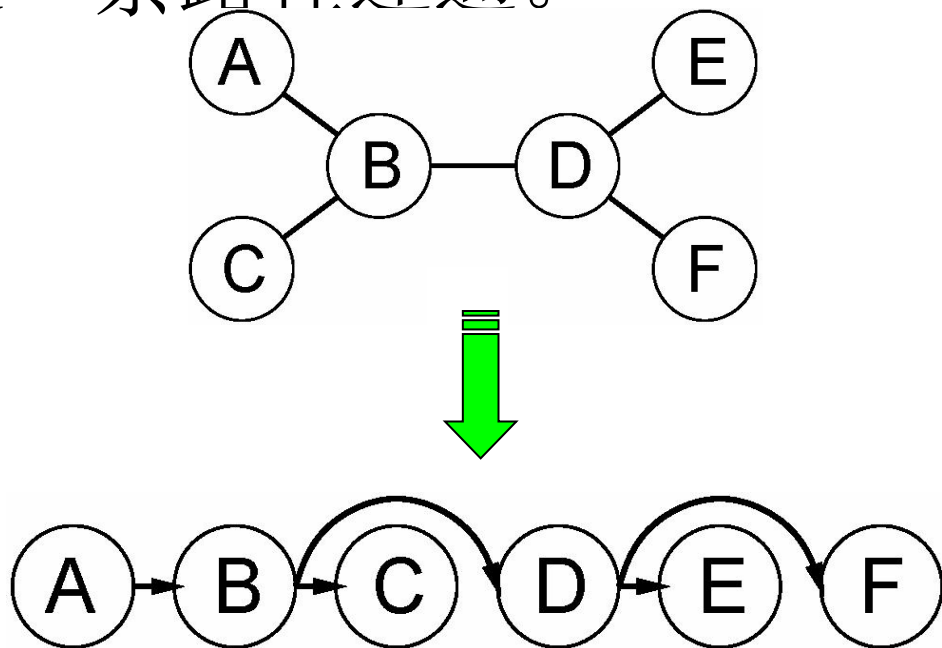
- Mini-conflict算法对许多CSP都惊人地有效，尤其是给定了合理的初始状态时。
 - 例如，对于八皇后问题，如果不计算皇后的初始位置，算法的运行时间大体上独立于问题的大小。
 - 百万皇后问题，平均50步（给定了初始赋值后）
- 局部搜索算法的另一个优势是当问题改变时可用于联机设置。
 - 这在调度问题中尤其重要。
 - 例如，恶劣天气导致航班日程表要修改，总是希望以最小改动来修改日程。
- 很容易CSP问题转换成最优化问题。这样，爬山法、模拟退化和遗传算法等都可以用来最优化目标函数。

- 约束满足问题简介
- 回溯搜索的改进技术（针对约束满足问题）
 - 变量和取值顺序
 - 通过约束传播信息
- 局部搜索
- 问题的结构

- 问题的结构，如约束图表示的，可利用来找到问题的解。
- 假设一个CSP问题的变量个数为 n ，所有变量的值域大小 $\leq d$ ，则问题的可能的完全赋值的数量为 $O(d^n)$ 。
- **分解方法**：将约束图分解为**连通域的并集**以降低问题的复杂度。
 - 例子：澳大利亚地图中Tasmania与大陆不相连。
- 假设每个连通域对应一个子问题 CSP_i 。如果赋值 S_i 是 CSP_i 的一个解，则 $\bigcup_i S_i$ 是 $\bigcup_i CSP_i$ 的一个解。
 - 假设总计有 n 个变量，每个 S_i 有 c 个变量，则共计 n/c 个子问题，每个子问题最多花费 d^c 步工作，则总工作量为 $O(d^c n/c)$ 。

约束图是树

- 问题：完全独立的子问题很少见。
- 考虑约束图是树的情景，即任何两个变量最多通过一条路径连通。



- 求解算法：

1. 任选一个节点作为根节点，从根节点到叶节点按顺序排列，每个节点的父节点都在它前面。按顺序把它们标为 $X_1, \dots, X_n$ 。
2. 令 j 从 n 到 2 ，在弧 (X_i, X_j) 上应用弧相容算法，其中 X_i 是 X_j 的父节点，从 $\text{DOMAIN}[X_i]$ 中删除必要的值。
3. 令 j 从 1 到 n ，赋给 X_j 与 X_i 的值相容的值。

✓ 第2步之后的CSP是有向弧相容的；

- 被删掉的值不会危及已处理过的弧的相容性。

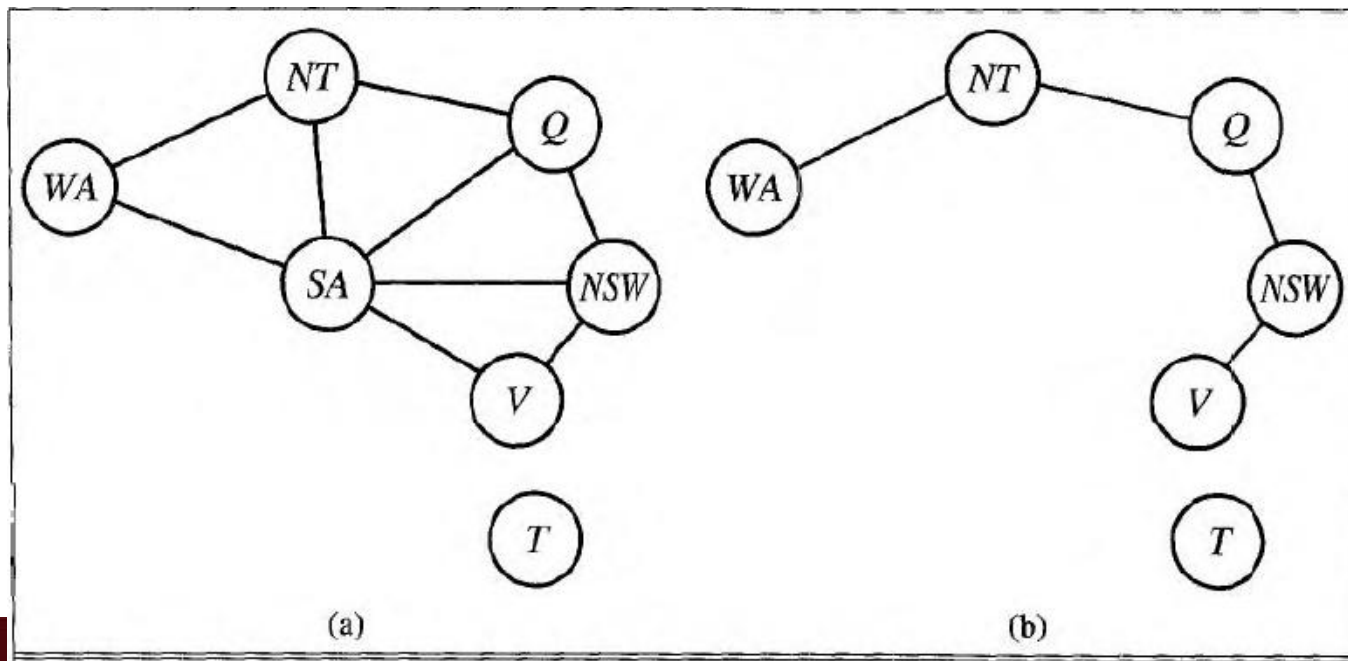
✓ 第3步，赋值不需要回溯。

✓ 时间复杂度： $O(nd^2)$ 。

- 将一般的约束图简化为树形式，有2中基本方式：
 - 基于删除节点的。
 - 基于合并节点的。

基于删除节点的方式

- 先对某些变量赋值，使剩下的变量构成一棵树。
- 例：澳大利亚的约束图，给SA赋值，并从其它变量的值域中删去与该值不相容的值。

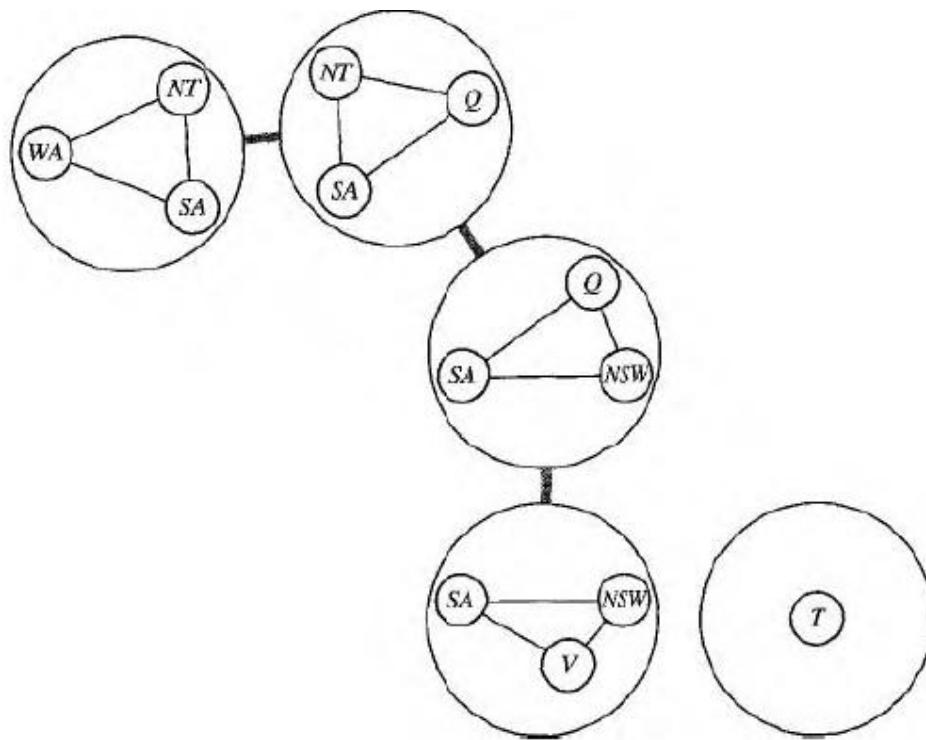


基于删除节点方式的一般算法

1. 从 $\text{VARIABLES}[\text{csp}]$ 中选择一个子集 S ，使得约束图在删除 S 之后成为一颗树。 S 称为 **环割集**。
 2. 对满足 S 所有约束条件的 S 中变量的 **每个可能赋值**:
 - a) 从 **剩余变量** 的值域中删除与 S 的赋值不相容的值;
 - b) 如果去掉 S 后的剩余 CSP 有解，把 **解和 S 的赋值** 一起返回。
- 如果环割集的大小为 c ，那么总的运行时间为 $O(d^c(n-c)d^2)$ 。
 - 寻找最小环割集是个 **NP 难题**。
 - 采用一些已有的高效近似算法，即 **割集调整**。

基于合并节点的方式

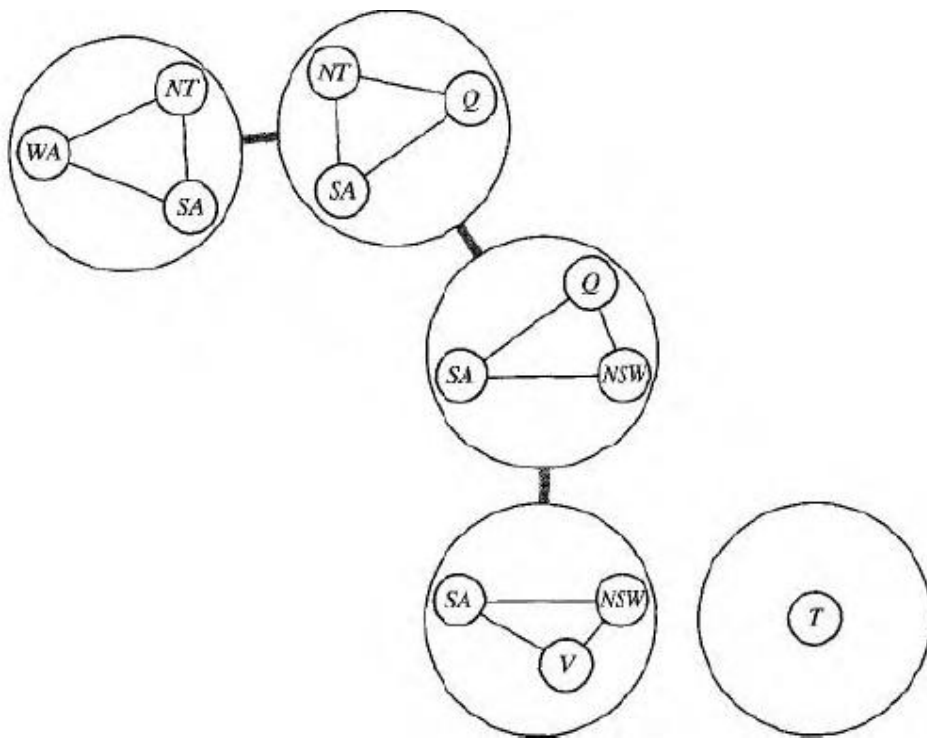
- **基于合并节点的方式**：建立在构造约束图的**树分解**的基础上，把约束图分解为相关联的子问题集。
 - 独立求解每个子问题
 - 合并结果



澳大利亚地图着色问题的约束图的分解

- 一个树分解须满足的条件：
 1. 每个原始问题中的变量至少在一个子问题中出现。
 2. 如果两个变量在原问题中由一个约束相连，那么它们至少同时出现在同一个子问题中（连同约束）。
 3. 如果一个变量出现在树中的两个子问题中，它必须出现在连接这些子问题的路径上的所有子问题里。
 - 任何给定的变量在每个子问题中必须取值相同。

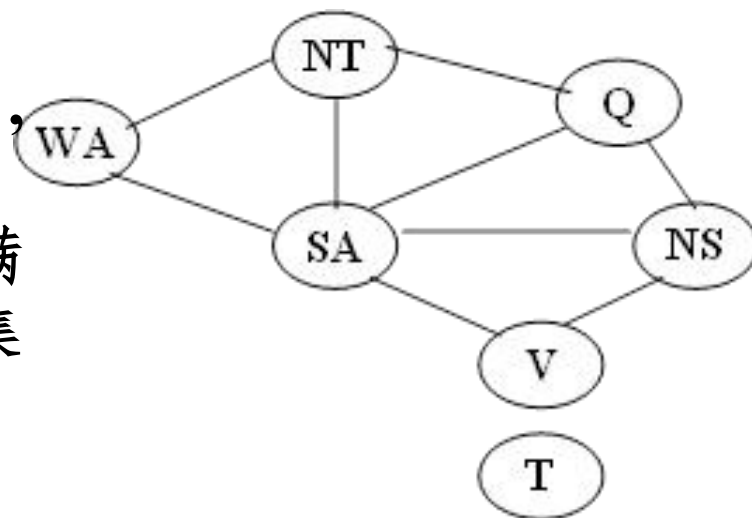
- 从各个子问题的解得到全局的解：把每个子问题视为一个大变量，它的值域是问题所有可能的解的集合。
 - 例如：最左边的子问题，有3个变量，有6个解。



- 一个给定的约束图允许有**多种**树分解。
 - 在选择不同分解的时候，目标是分解出来的子问题越小越好。
- 图的树分解的**树宽**比**最大的子问题的大小**少1。
- 图自身的树宽定义为它**所有可能树分解的最小树宽**。
- 如果一个图的树宽为 w ，那么这个问题的时间复杂度是 $O(nd^{w+1})$ 。
 - 因此，约束图树宽有界的CSP问题是多项式时间内可解的。
 - 不幸的是，找到最小树宽的树分解是NP难题。
 - 有一些启发式方法。

- 约束满足问题（CSP），用图表示它的结构
- CSP问题的回溯搜索；深度优先搜索的一种形式
 - 从三个方面优化搜索
- 约束满足问题的局部搜索
 - 最小冲突启发式
- 问题的结构
 - 割集调整将问题化为树状结构
 - 树分解将问题化为子问题的树

- 澳大利亚地图染色问题，是指用红绿蓝3色标出各省，且相邻者颜色不同。下图给出了澳大利亚地图着色问题的约束图。
 - ① 假设当前所有变量都还没有赋值，按照度启发式，请问应该选择哪一个变量进行赋值？
 - ② 假设已有不完全赋值 $\{WA=red, NT=green\}$ ，下面需要在SA或Q这两个变量之间选择一个进行赋值。如果采用MRV（最少剩余值）启发式来选择变量，请问应该选择哪一个？
 - ③ 假设已有不完全赋值 $\{Q=red, NSW=green, V=blue, T=red\}$ 。此时，给SA赋值将发现无法满足约束条件。请问SA的冲突集是什么？





合肥工业大学
HEFEI UNIVERSITY OF TECHNOLOGY

Q/A?



合肥工业大学数据挖掘与智能计算“千人计划”研究团队
HFUT Data Mining and Intelligent Computing Laboratory